

Explainable Deep Learning: Comparative Analysis of Grad-CAM and LIME

ACIT4620 Exame - Group C1
Wordcount - 7200

Max Dyrø Finanger
OsloMet
madyr3382@oslomet.no

Felix Boehm
OsloMet
feboe8099@oslomet.no

Sebastian Skrøvseth Haugen
OsloMet
sehau5609@oslomet.no

Thomas Heim
OsloMet
thhei7342@oslomet.no

Jesse Kyomuhendo Tibamwenda
OsloMet
jetib6962@oslomet.no

Abstract—Deep learning models often operate as black boxes, producing predictions through complex mathematical transformations without providing insight into their decision-making process. Explainable artificial intelligence(XAI) methods attempt to explain the output of the model. We used Grad-CAM and LIME, in order to explain how a fine-tuned ResNet-50 model classified images from the CIFAR-10 dataset. Grad-CAM leverage gradient information to identify discriminative regions, while LIME use input perturbation to generate local explanations. The model is initialized with ImageNet pretrained weights and all layers are fine-tuned for CIFAR-10 classification. Using balanced class representation across training, validation, and test sets, our model achieves a mean 93.3% classification accuracy. We evaluate the quality of explanations using Intersection over Union (IoU), achieving a mean IoU of 0.364. Our per-class analysis reveals varying agreement levels across different classes, with some classes showing high spatial overlap between methods while others had lower consensus. Our analysis demonstrates that Grad-CAM and LIME provide complementary explanations. This findings highlights the importance of using multiple XAI methods to gain comprehensive understanding of model behavior.

Index Terms—Explainable AI, Artificial Intelligence, Grad-CAM, LIME, Image Recognition, Computer Vision, LIME

I. INTRODUCTION

Deep learning has become the standard approach for image classification, where models learn to map raw pixel values directly to semantic labels. Convolutional neural networks (CNNs) in particular have shown strong performance on many benchmark datasets and are now widely used in real-world applications such as medical imaging, surveillance and autonomous systems [9]. However, even when these models achieve high accuracy, their internal decision processes remain difficult to understand. As a result, this black-box behaviour makes it difficult to know whether a model is relying on the right visual cues, or whether it has learned shortcuts and dataset biases that might fail in new situations [8].

Explainable AI (XAI) aims to address this problem by providing human-understandable explanations for model predictions [5], [6]. In computer vision, these explanations often

highlight which parts of an image were most important for a given decision. These explanations help practitioners verify that a classifier focuses on the relevant regions, debug unexpected errors, and build trust when models are deployed in safety-critical contexts. However, without this level of transparency, it becomes difficult to justify the use of deep learning systems in domains where accountability and interpretability are essential.

In this project we focus on two widely used XAI methods, namely Gradient-weighted Class Activation Mapping (Grad-CAM) and Local Interpretable Model-agnostic Explanations (LIME). Grad-CAM is a gradient-based method that produces a coarse heatmap over convolutional feature maps, showing where the network is focusing when it makes a prediction [5]. LIME is a perturbation-based method that explains individual predictions by turning image regions on and off and fitting a simple, local surrogate model [6]. Both methods are commonly used in practice. However, they are based on very different principles, and it is not obvious whether they highlight similar or different regions of an image.

We use the CIFAR-10 dataset and ResNet-50 model as the basis for our study. CIFAR-10 is a balanced and widely used benchmark that contains small, low-resolution images from ten everyday object classes [1]. Therefore, it is well suited for experiments that need to be reproducible and computationally manageable, while still being relevant for general-purpose image classification. ResNet-50 is a standard CNN architecture that is available with pretrained ImageNet weights and is frequently used as a backbone in computer vision [2]. In addition, using CIFAR-10 and ResNet-50 together allows us to fine-tune a strong classifier on accessible hardware and then systematically analyse its explanations.

The main motivation of this work is to understand how much Grad-CAM and LIME actually agree when explaining the same model, and how this agreement behaves across different classes. However, much of the existing XAI literature evaluates methods visually [3], [7] or in specific domains, but there is less focus on quantitative comparisons on general

benchmarks like CIFAR-10. Moreover, it is unclear whether moderate overlap between explanations should be seen as noise, disagreement, or useful complementarity. Therefore, this study investigates the extent to which Grad-CAM and LIME provide spatially consistent explanations for a ResNet-50 classifier on CIFAR-10, and how this consistency differs across object classes.

Based on this research aim, our study is guided by the following objectives:

- Train and fine-tune a ResNet-50 model on CIFAR-10 using a reproducible setup
- Generate Grad-CAM and LIME explanations for a stratified subset of correctly classified test images
- Convert both explanations into comparable masks and compute a quantitative agreement measure between them
- Analyse class-specific patterns in agreement scores
- Discuss how these results can guide practitioners in choosing and combining XAI methods

The rest of the paper is organized as follows. Section 2 reviews related work on deep learning for image classification, pretrained models, existing XAI methods, and remaining research gaps. Section 3 describes our method and materials, including the computational environment, dataset, model architecture, training procedure, XAI implementations, and comparison metric. Section 4 presents the experimental results including overall model performance and the quantitative XAI analysis. Section 5 discusses these findings, the strengths and limitations of our approach, and potential applications of combined Grad-CAM and LIME explanations. Finally, section 6 concludes the paper and outlines directions for future work.

While Grad-CAM and LIME are widely used to explain CNN predictions, their spatial agreement remains underexplored, raising concerns about explanation reliability. This study addresses the following research question:

Research Question: *To what extent do Grad-CAM and LIME provide spatially consistent explanations for a ResNet-50 classifier on CIFAR-10, and how does consistency vary across object classes?*

To answer this question, we fine-tune ResNet-50 on CIFAR-10, generate explanations using both methods, and quantify spatial overlap using Intersection-over-Union (IoU) across all ten classes.

II. LITERATURE REVIEW

A. Deep learning for image classification

Deep learning and Convolutional Neural Networks (CNNs), changed how image classification is done by learning from the hierarchical feature representations directly from the images raw pixel data, which overcame the limitations of how traditional, handcrafted feature methods are done. Examples are models like AlexNet [1] and VGG [2] which established the deep architecture paradigm which led to large performance breakthroughs on large-scale datasets such as on ImageNet [3]. Benchmarking performance is often done on datasets like CIFAR 10 [4]. This gives a standard testbed for evaluating new

architectures and training techniques. Another innovation in the field focused on managing the vanishing gradient problem and getting better efficiency, which lead to the development of architectures like ResNet [5]. Today models like this are standard for doing complex visual recognition tasks.

B. Pretrained Models

When there is limited label data for a task, using transfer learning provides an effective solution. This involves taking a model that is pre-trained on a vast, general dataset, such as for example ImageNet and applying it to new, specific tasks. Earlier work has shown that deep neural networks transfer features well across domains [6]. The pre-trained model, which has low-level features such as edges and textures, also has high-level semantic features from a vast dataset and this provides a good starting point with the advantage of reducing training time and improving the generalization.

ResNet stands for Residual Network [5] architecture and the ResNet-50 variant is a popular pretrained model in computer vision. ResNet introduced residual connections, which allows the output of a layer to be added to the input of a later layer, enabling the training of a much deeper networks without degradation in performance. This architecture makes ResNet-50 an optimal backbone for transfer learning in various domains such as medical imaging. For example brain tumor classification [7] and general object recognition.

C. Explainable AI Methods

Explainable AI (XAI) methods are used to increase trust by showing what features a model is looking at to make a decision. Different XAI methods have different scopes. A local scope focuses on explaining a single prediction while a global scope aims to explain the entire model behaviour. There are also model agnostics which work on any model while model-specific specific which only work on one model. A comprehensive review of XAI in computer vision shows the field is expanding quickly [8].

Gradient-weighted Class Activation Mapping (Grad-CAM) is a model-specific, local XAI technique. It works by using the gradients of the target class flowing into the last convolutional layer to produce a coarse localization visualization. This heatmap shows which regions of the input image were most influential in the models prediction for that specific class. This is an effective way of providing an attention-like visualization in convolutional networks [9].

Local interpretable model-agnostic explanations (LIME) is a model-agnostic, local XAI model that works by perturbing the input image. It does this by turning on/off superpixels and trains a simple interpretable model locally around the specific instance being explained. The weights of this simple model reveal which regions of the image contributed strongly to the prediction being made by the deep learning model [10].

D. Gaps in Research

XAI have been advancing, but there are still several gaps that remain, which the proposed work tries to address.

While papers like Yoon and Lin [7] perform comparative studies of LIME and Grad-CAM, these are very specialized on medical imaging. XAI methods and the consistency of their explanations can vary significantly across different model architectures and image domains.

Our contributions to explainable AI are that the project provides a direct, side-by-side comparative evaluation of LIME and Grad-CAM on a standard, general-purpose benchmark (CIFAR-10). In this we will systematically compare the resulting feature importance (LIME masks) and attention regions (Grad-CAM heatmaps) to assess their consistency across different classes, which can give us insight into their agreement and disagreements on common visual data.

The literature also shows the risk of deep learning models relying on irrelevant features, often due to dataset bias. Geirhos [11] describes this as "shortcut learning" and shows neural networks often exploit unintended background correlations rather than actually learning the object features. For instance, Bird and Lotfi [12] showed that a model focused on the subtle background changes instead of the central object to detect AI-generated images. Similar to this, Yoon and Lin [7] also observed misclassifications where models focused on irrelevant structures like noise or boundaries in MRI scans.

We compare LIME and Grad-CAM as a diagnostic tool to identify and visualize unreliable feature reliance within a pre-trained CNN classifying CIFAR-10 images. Our methodology aims to discover whether the model focuses on the expected object such as an airplane or external features such as the sky or the background. We can find potential biases in the ResNet architecture when applied to 32x32 images.

If you look at existing studies on XAI they rely on specialized datasets, which can make it difficult to replicate the results. In contrast, we have used the publicly available CIFAR-10 dataset with a standard, open-source pretrained CNN such as ResNet-50. This means that our work is fully reproducible and accessible, showing a clear case study for interpreting the model behaviour using XAI methods. The results using LIME and Grad-CAM offer intuitive and understandable insights that helps with analysis and education in explainable machine learning.

III. METHOD AND MATERIALS

A. Computational Environment

All experiments were conducted using Python and PyTorch. Model training and evaluation were performed on an Apple MacBook Pro with Apple M1 chip and 16 GB unified memory (Apple Silicon). No external GPU was used. Fine-tuning the model over 20 epochs took around 6 hours. We chose this hardware as we did not have access to expensive GPUs, making it accessible to other researchers to reproduce our findings, as this does not require the newest CPUs. The training process remained stable throughout, with no hardware-related failure or memory overflow issues encountered during our experiments.

B. Code and Model Availability

All source code and preprocessing scripts used in this paper are provided in the appendix and are publicly available on GitHub: <https://github.com/maxfinanger/4620-Exam>.

The fine-tuned ResNet-50 model weights are hosted on the Hugging Face Hub due to size constraints and can be accessed at: <https://huggingface.co/GithubBro/resnet50-finetuned-cifar10-classification/tree/main>.

Both repositories are publicly accessible to support reproducibility and further research.

C. Dataset and Preprocessing

We use the CIFAR-10 dataset [4], a widely used benchmark in computer vision research. The dataset contains 60,000 color images (32x32 pixels) distributed equally across ten object classes: airplane, automobile, bird, car, deer, dog, frog, horse, ship, and truck. Each class has exactly 6,000 images, ensuring balanced representation across all categories [4]. The dataset's small image size and diverse object categories make it good for evaluating the model's reasoning for image classification. Also, the dataset is not too big, so we can use it to fine-tune the model on our current hardware without any problems.

We divided the dataset into three parts using random splitting with seed 42 for reproducibility. The original 50,000 training samples were split into 40,000 training images (80%) and 10,000 validation images (20%). The original 10,000 test samples remained as our held-out test set. This three-way split followed standard machine learning practice: we trained on the training set, tuned hyperparameters using the validation set, and reported final performance on the test set that the model never sees during development. As standard practice, this is done to prevent overfitting of the model.

Since ResNet-50 expects 224x224 pixel inputs, but CIFAR-10 provides 32x32 images, we upsampled all images using bilinear interpolation. While this introduces some artificial smoothness, it allows us to use pretrained weights effectively. We then normalize pixel values using ImageNet statistics: RGB means [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225]. This normalization is crucial, ResNet-50 was originally trained on ImageNet using these exact values, so maintaining consistency ensures the pretrained features transfer effectively [5].

For training images only, we applied data augmentation to artificially expand the dataset and reduce overfitting. First, we apply horizontal flipping with 50% probability. Second, we use random cropping with 4-pixel padding for slight position variations. Third, we apply color jittering that randomly adjusts brightness and contrast by $\pm 20\%$. The combination of these techniques effectively increases our training data by 10-15 times. Validation and test images receive only resizing and normalization without any augmentation.

We organized images into batches of 64 for balancing GPU memory efficiency. Training batches are shuffled randomly at each epoch to prevent the model from learning patterns based on data order. The 40,000-sample training set required 624 batches per epoch. Validation used the same batch size

across 157 batches. This batch configuration maintained a consistent computational pattern across all evaluation phases while preventing memory overflow. Validation and test batches maintain fixed order for consistent validation.

D. Model Architecture and Fine-Tuning Strategy

We use ResNet-50 [5] as our base model, loading it with ImageNet-1K_V1 pretrained weights. ResNet-50 has already learned useful visual features from 1.28 million ImageNet images over 1,000 categories, giving us a strong starting point.

We modify only the final classification layer to adapt ResNet-50 to the CIFAR-10 classes. The original ResNet-50 has 2048 features feeding into a classification layer for 1000 ImageNet classes. We replace this with 2048 features with a dropout layer that randomly deactivates 50% of connections during training to prevent overfitting, followed by a linear layer producing 10 outputs, where we have one per CIFAR-10 class.

We use full fine-tuning, meaning all model parameters remain trainable during training. This allows the entire model to adapt specifically to CIFAR-10’s characteristics while retaining useful patterns from ImageNet.

For generating visual explanations, we target layer4[-1], which is the final building block in ResNet-50’s fourth stage. This layer produces 7×7 feature maps with 2,048 channels, providing a good balance between spatial detail and semantic understanding. This layer is therefore a good choice for conducting our experiment.

E. Training Configuration and Regularization

We train using the Adam optimizer from the PyTorch library with an initial learning rate of 0.0005 and weight decay 5×10^{-4} . Learning rate of 0.0005 was selected as half the standard Adam default of 0.001 to enable stable fine-tuning of pretrained weights. The Adam optimizer adapts learning rates automatically for each parameter, making training more stable. The weight decay term implements L2 regularization, discouraging large parameter values that often indicate overfitting.

Our loss function combined standard cross-entropy with label smoothing ($\alpha = 0.1$). Label smoothing prevents overconfidence by assigning 91% probability to the correct class and distributing the remaining 9% across classes.

We implemented ReduceLROnPlateau from the PyTorch library, which is a learning rate scheduler that monitors validation loss after each epoch. When validation loss stops improving for three consecutive epochs, the scheduler reduces the learning rate by a factor of 0.1. This adaptive strategy allows large updates early in training while enabling more precise optimization later. Additionally, we applied gradient clipping maximum norm of 1.0 to prevent extremely large gradient values that could destabilize training.

During validation, we tracked not only overall accuracy but also per-class accuracy to detect situations where the model improves on most classes, while not for others. We only save the model weights when overall validation accuracy improves and no individual class accuracy drops by more than 5% from

the previous best value. This prevents the model from getting an overall good accuracy by sacrificing accuracy for other minority classes. If validation accuracy doesn’t improve for 5 consecutive epochs, training stops early to prevent overfitting.

F. Grad-CAM Implementation

Grad-CAM creates visual explanations showing which image regions most influence the model’s predictions. We implemented Grad-CAM using a Python class with six steps.

- 1) **Forward Pass:** We feed the input image through ResNet-50 and capture the activations (feature maps) from layer[-1] using a forward hook. This produces a tensor of shape $1 \times 2048 \times 7 \times 7$, representing 2,048 different feature detectors each producing a 7×7 spatial map.
- 2) **Backward Pass:** We select the target class score from the model’s output and compute gradients using back-propagation. A backward hook captures these gradients ($1 \times 2048 \times 7 \times 7$), showing how much each feature influences the prediction.
- 3) **Weight Calculation:** We multiply each feature map A_k by its importance weight α_k and sum across all 2,048 channels: $L_{\text{Grad-CAM}} = \sum_k \alpha_k A_k$. This weighted sum emphasizes feature detectors that are strongly activated for the predicted class while downweighting irrelevant detectors.
- 4) **Weighted Combination:** Each feature map is multiplied by its corresponding importance weight, and all 2048 weighted maps are summed to create a single 7×7 importance map.
- 5) **ReLU Activation:** We apply ReLU to keep only positive values. This focuses our visualization on features that support the prediction rather than features that argue against it.
- 6) **Visualization:** Values are normalized to the $[0, 1]$ range using min-max scaling. The 7×7 map is resized to 224×224 , where matplotlib automatically upsamples the 7×7 grid to match the image dimensions using bilinear interpolation. The final heatmap overlays the original image using a colormap, where warm colors (red/yellow) indicate high importance and cool colors (blue) indicate low importance. This creates intuitive visualizations where red/yellow areas highlight decision-critical regions.

G. LIME Implementation

LIME creates explanations by fitting simple linear models that approximate the neural network’s behavior locally around a specific prediction. We implement LIME using the official lime-library and get the core logic from the library for logic-segmentation, perturbation, regression, feature selection, and visualization. Our own implementation is minimal. We provided the model prediction function `batch_predict`, which LIME calls internally for each perturbed image. This makes our code concise and focused, while the LIME library handles all the complex steps under the hood.

Below is an explanation of what happens inside LIME when function `batch_predict` is called.

- 1) **Segmentation:** The Quickshift algorithm divides the 224×224 input image into approximately 50–80 superpixels. Each superpixel groups neighboring pixels with similar colors into perceptually meaningful regions.
- 2) **Perturbation:** LIME creates 1,000 modified versions of the original image. Each version randomly turns superpixels “on” (showing original colors) or “off” (replaced with neutral gray), generating variations around the original image.
- 3) **Prediction:** Our custom `batch_predict()` function processes all 1,000 perturbed images efficiently. It converts images to tensors, applies ImageNet normalization, processes images in batches of 64, and returns probability distributions for all ten classes.
- 4) **Linear Model Fitting:** LIME fits a weighted linear regression model where perturbations closer to the original image receive a higher weight. The model uses L1 regularization to encourage sparse solutions with fewer superpixels, making explanations easier to interpret.
- 5) **Feature Selection:** LIME ranks all superpixels by their linear regression coefficients and selects the top $K = 10$ as the final explanation. We use `positive_only=True` to show only superpixels that increase confidence in the predicted class. The final visualization uses `mark_boundaries` from `scikit-image` to draw boundaries around selected superpixels on the original image.

H. Quantitative Comparison Methodology

To objectively measure agreement between Grad-CAM and LIME, we used Intersection over Union (IoU), a standard metric for comparing overlap between regions.

We converted both explanations to binary masks representing important vs. unimportant regions. For Grad-CAM, we applied a threshold at the 70th percentile, marking pixels with heatmap values in the top 30% as important (value = 1) and all others as unimportant (value = 0). This threshold was chosen to focus on the most activated regions while filtering out background noise. For LIME, the top $K = 10$ selected superpixels were marked as important (value = 1), with the remaining regions marked as unimportant (value = 0). The value $K = 10$ was selected to balance interpretability with coverage, ensuring explanations highlight key features without excessive fragmentation. With both methods producing 224×224 binary masks, IoU is computed as:

$$\text{IoU} = \frac{|\text{Mask}_{\text{Grad-CAM}} \cap \text{Mask}_{\text{LIME}}|}{|\text{Mask}_{\text{Grad-CAM}} \cup \text{Mask}_{\text{LIME}}|}$$

The numerator counts pixels where both Grad-CAM and LIME agree a region is important (both masks equal 1). The denominator counts all pixels that either method considers important (at least one mask equals 1). For example, if Grad-CAM highlights 5,000 pixels and LIME highlights 4,000 pixels, with 2,500 pixels overlapping, then $\text{IoU} = 2,500 /$

$(5,000 + 4,000 - 2,500) = 2,500 / 6,500 = 0.385$. IoU ranges from 0 (no overlap, complete disagreement) to 1 (perfect overlap, complete agreement).

We used sampling to ensure balanced evaluation across all object categories. Rather than randomly sampling 20 images that might miss some classes, we deliberately select exactly 2 images per class using seed 42 for reproducibility. This guarantees representation of all ten CIFAR-10 categories. For each of the 20 samples, we generated summary statistics and per-class average IoU to identify category-specific patterns.

We interpret results using three thresholds:

- $\text{IoU} > 0.40$: high agreement, indicating both methods identify similar important regions.
- $\text{IoU} 0.30\text{--}0.40$: moderate agreement, indicating partial overlap with some differences.
- $\text{IoU} < 0.30$: low agreement, indicating methods focus on different regions.

These thresholds help us categorize explanation consistency and identify which object class produces reliable, agreed-upon explanations, and which one is ambiguous.

IV. RESULTS

Training ResNet-50 on CIFAR-10 with the regularization strategy achieved convergence within 20 epochs. Figure 1 shows the training and validation learning curves throughout the training process.

A. Model Training Performance

The training and validation loss curves have a clear downward trend. During epochs 1-5, both losses decreased sharply from their initial values, with training and validation curves having the same parallel paths. The intermediate phase (epochs 6-10) showed continued training loss decline while validation loss fluctuates, but overall has a downward trend. At epoch 10, the validation loss temporarily converged with the training loss before diverging again. The final phase (epochs 11-20) shows continued gradual training loss reduction while validation loss plateaued with minimal fluctuation after around epoch 14. By epoch 20, a clear gap has emerged between the training loss baseline and the higher validation loss plateau.

The training and validation accuracy showed a similar pattern to the loss trajectories. Training accuracy increased steeply during epochs 1-5, then continued a smooth upward progression through the remaining epochs, and reached a value in the upper range by epoch 20. Validation accuracy followed a more variable path, with steady improvement during epochs 1-5, oscillatory behavior during epochs 6-10, including a spike around epoch 10, followed by stabilization. After epoch 14, validation accuracy maintained a constant level without further improvement despite ongoing training accuracy increase. After epoch 15, validation accuracy stays more or less flat, while training accuracy is still increasing.

B. Test Set Evaluation

Final evaluation on the held-out test set yielded 92.60% accuracy with a test loss of 0.675. Figure 2 and Table I show

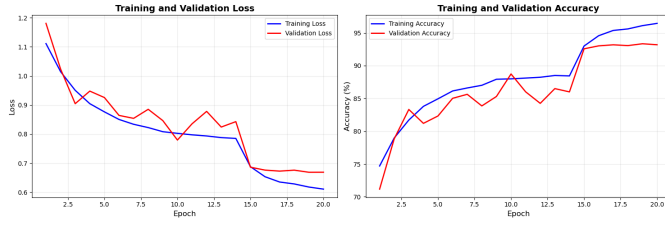


Fig. 1. Training and validation learning curves for ResNet-50 on CIFAR-10.

the detailed per-class performance metrics from the confusion matrix analysis.

TABLE I
PER-CLASS TEST SET ACCURACY

Class	Accuracy (%)	Correct/Total
Airplane	94.7	947/1000
Automobile	97.3	973/1000
Bird	90.0	900/1000
Cat	85.8	858/1000
Deer	93.7	937/1000
Dog	85.5	855/1000
Frog	95.3	953/1000
Horse	96.8	968/1000
Ship	94.8	948/1000
Truck	95.1	951/1000
Overall	92.6	9260/10000

The confusion matrix (Fig. 2) revealed distinct accuracy patterns across classes. Mechanical objects achieved accuracies above 94%: automobiles (97.3%), horses (96.8%), trucks (95.1%), frogs (95.3%), ships (94.8%), and airplanes (94.7%). Deer reached 93.7%. Animal classes with fur textures showed lower performance: birds (90.0%), cats (85.8%), and dogs (85.5%).

Off-diagonal elements in the confusion matrix revealed systematic misclassification patterns. Cats were misclassified as dogs in 56 cases, deer in 22 cases, birds in 19 cases, and frogs in 16 cases. Dogs were misclassified as cats in 84 cases, deer in 19 cases, birds in 11 cases, and horses in 8 cases. The cat-dog confusion occurred bidirectionally, with 56 cats misclassified as dogs and 84 dogs misclassified as cats.

Deer were confused with cats in 22 cases, horses in 16 cases,

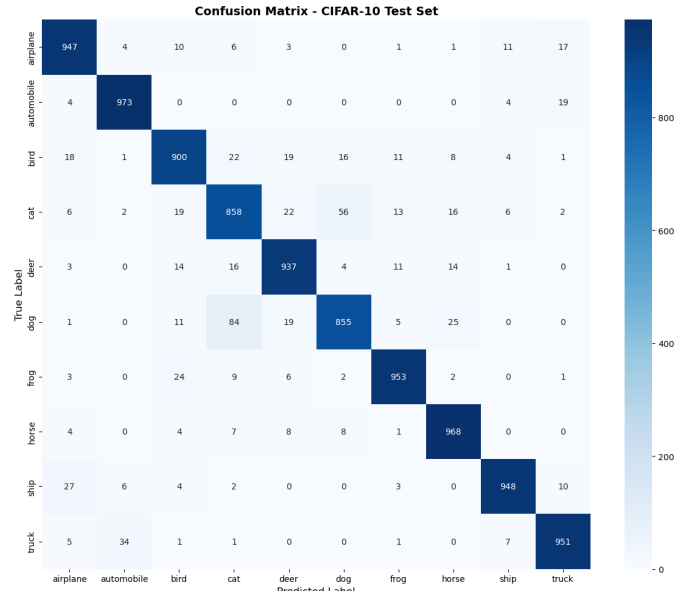


Fig. 2. Confusion matrix for CIFAR-10 test set predictions.

birds in 14 cases, dogs in 8 cases, and frogs in 4 cases. Birds were misclassified as frogs in 24 cases, cats in 22 cases, deer in 19 cases, airplanes in 18 cases, dogs in 11 cases, and horses in 8 cases. Bird-airplane confusion occurred in 18 cases.

Artificial classes showed minimal cross-class confusion. Automobiles had only 4 misclassifications to airplanes and 19 to trucks. Airplanes were confused with ships in 11 cases and trucks in 17 cases. Ships were confused with airplanes in 27 cases and trucks in 10 cases. Trucks were confused with automobiles in 34 cases and airplanes in 5 cases.

Horses were misclassified as deer in 16 cases, cats in 7 cases, birds in 4 cases, dogs in 1 case, and frogs in 1 case. Frogs were confused with birds in 24 cases, dogs in 11 cases, deer in 6 cases, ships in 3 cases, horses in 2 cases, and cats in 1 case.

C. Explainable AI Analysis: Grad-CAM vs. LIME

We conducted a comprehensive XAI analysis covering all ten CIFAR-10 classes (2 samples per class, 20 total samples). Figures 3 and 4 present side-by-side visual comparisons of Grad-CAM and LIME explanations for the test images.

1) *Method-Specific Characteristics:* **Grad-CAM** produced smooth, continuous heatmaps highlighting broad spatial regions. For mechanical objects (automobiles, ships, trucks), Grad-CAM consistently highlighted the entire object body, focusing on central structural features. Automobile explanations showed strong activations on the vehicle chassis and wheel regions. Ship explanations highlighted distinct parts of the boat structure. Truck explanations emphasized the front of the truck and the cargo compartment.

For animals, Grad-CAM heatmaps showed different spatial patterns based on different features. Frogs had concentrated central activation on body regions. Horses and deer showed activation patterns following body contours with emphasis on

Grad-CAM vs. LIME Comparison

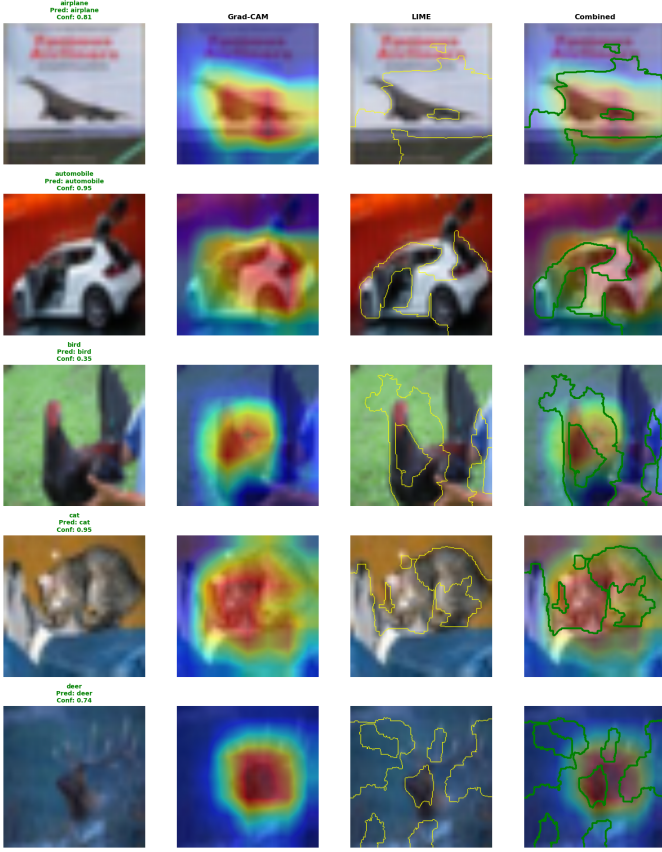


Fig. 3. Grad-CAM vs. LIME comparison for classes: airplane, automobile, bird, cat, and deer. From left to right: original image with prediction, Grad-CAM heatmap overlay, LIME superpixel boundaries, and combined visualization.

torso and face. Cats and dogs had a more diffuse activation pattern distributed across the body and facial features. Birds showed compact activation in the neck rather than distributed across feather textures.

LIME generated discrete superpixel-based explanations focusing on perceptually meaningful image regions. For mechanical objects, LIME concentrated on geometric boundaries and structural edges. Automobile explanations focus on the vehicle body and wheel areas. Ship explanations focused on the distinctive part of the boat structure. Truck explanations distributed selection across the front of the truck, cargo space, and wheel regions.

For animals with textured surfaces, LIME selected more fragmented superpixel sets. Frogs showed selections distributed across skin texture patterns and body outlines. Cats and dogs exhibited highlighted regions around fur texture patterns, facial features, and body contours. LIME’s superpixel boundaries occasionally fragmented the coherent object regions, particularly for geometrically simple objects like airplanes and ships.

Horse explanations followed body shape to a certain extent, but they also covered the background. Deer focused on the

Grad-CAM vs. LIME Comparison

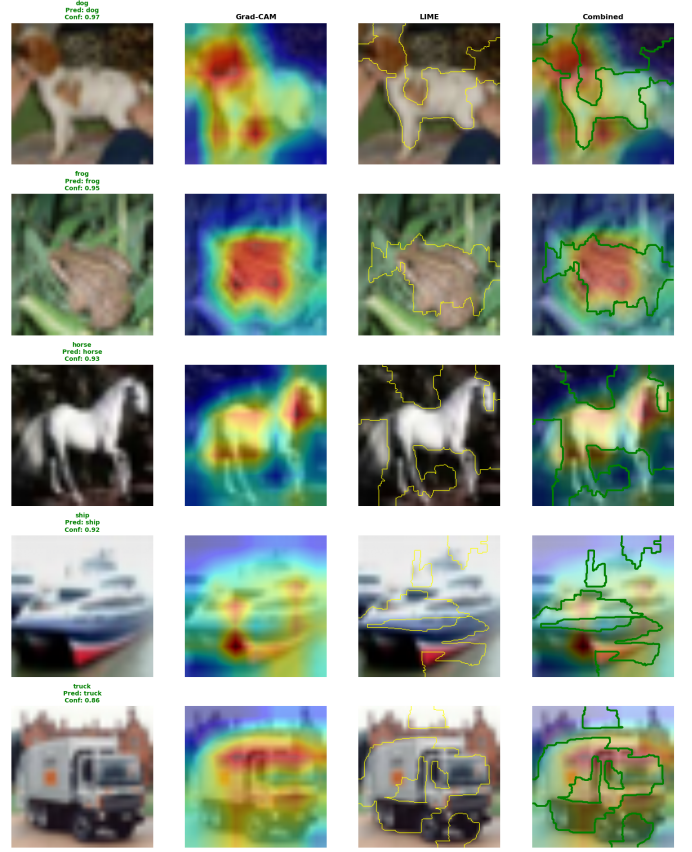


Fig. 4. Grad-CAM vs. LIME comparison for classes: dog, frog, horse, ship, and truck. From left to right: original image with prediction, Grad-CAM heatmap overlay, LIME superpixel boundaries, and combined visualization.

facial feature and antler structure, and the rest on the background. Bird focused on facial features and the body.

Combined visualizations (rightmost column in Figures 3 and 4) showed Grad-CAM heatmaps overlaid with LIME superpixel boundaries (green contours). High-agreement classes (automobile, frog, ship) showed special overlap between Grad-CAM and LIME. Moderate-agreement classes showed partial intersection between LIME and Grad-CAM. For the automobile, LIME superpixel boundaries aligned with the Grad-CAM peak activation region on the vehicle chassis. For frogs, LIME boundaries overlapped substantially with Grad-CAM. For ships, LIME boundaries for the ship body match with Grad-CAM structural emphasis.

Moderate-agreement classes displayed patterns that not always match up. For cats and dogs, LIME texture-based superpixels showed partial overlap with Grad-CAM broader activation regions. For airplanes, LIME’s wing selection showed moderate spatial overlap with Grad-CAM. Horses showed moderate overlap between methods, while trucks followed a similar pattern. Deer explanations showed weaker spatial correspondence between LIME features and Grad-CAM body region activations.

2) *Quantitative Spatial Agreement*: The Intersection over Union (IoU) analysis revealed moderate overall agreement between Grad-CAM and LIME (mean IoU = 0.392, median = 0.347, std = 0.095). Figure 5 shows the distribution of IoU scores and per-class spatial overlap metrics.

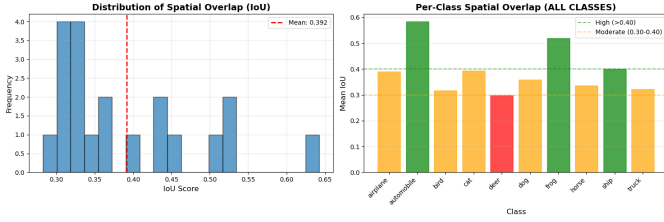


Fig. 5. IoU distribution and per-class spatial overlap analysis between Grad-CAM and LIME. Left: histogram of IoU scores across all samples. Right: mean IoU per class with color-coded performance levels (green: high agreement ≥ 0.40 , orange: moderate agreement 0.30-0.40, red: low agreement < 0.30).

Three distinct agreement categories emerged from the sampling (2 samples per class):

High Agreement (IoU > 0.40): Three classes achieved high spatial correspondence. Three classes achieved high spatial correspondence. Automobile showed the strongest agreement (IoU = 0.584), followed by frog (IoU = 0.519). Ship achieved borderline high agreement (IoU = 0.401). Both methods consistently identified similar spatial regions for these classes.

Moderate Agreement (IoU 0.30-0.40): Six classes fell into this category: airplane (IoU = 0.390), cat (0.394), dog (IoU = 0.358), horse (IoU = 0.336), bird (IoU = 0.317), and truck (IoU = 0.322). These classes demonstrated partial spatial overlap between Grad-CAM and LIME attributions.

Low Agreement (IoU < 0.30): Deer (0.298) showed the weakest spatial agreement. Grad-CAM highlighted broader body regions, while LIME identified scattered local features.

V. DISCUSSION

This discussion will focus on the methods and results generally, and specifically the accuracy, confusion patterns and findings. We will explain why some classes might be harder or easier for the model. We will discuss the strengths, limitations of our approaches and the possible future improvements we would make given the knowledge we have now. We will also reflect on the agreement between Grad-CAM and LIME.

1) *Interpretation*: The ResNet-50 algorithm in general has high accuracy, labeling 93.3 percent of all images correctly. Although the algorithm labels some image categories more precisely, Automobile images are correctly labeled in 97.3 percent of the cases. This is the most accurate category. Other categories proved to be more difficult, Cats and Dogs get 85.8 and 85.5. percent, respectively, and are the hardest categories to label for this algorithm. It is interesting that the algorithm struggled to differentiate the dogs and cats, most of the wrongly labeled cat images got labeled dog and vice versa. This might indicate that the model relies on textures and cues

from local patterns rather than global shapes to distinguish animals with similar pelts. A similar pattern emerges from the horses and deer, where horses most often are misclassified as deer, while deer most often gets misclassified as cats and then horses. Texture bias is a known phenomenon and has been documented when training CNNs on ImageNet [13]. The bias persists even after fine tuning, this indicates that the model excels at clear contoured structures objects, and struggles with classifying images that require a holistic understanding of the image in total. In other words there is a difference when classifying between different categories of images, our results show that some categories are easy for the model to classify with a high level of agreement between Grad-CAM and LIME, and high accuracy. While other categories still maintain a relatively high accuracy, but suffer from less agreeable features between Grad-CAM and LIME, and also have a higher level of false positives and false negatives.

2) *Grad-CAM vs LIME*: The moderate mean (IoU < 0.392), between Grad-CAM and Lime shows their complementarity. One might believe that using both these methods to be redundant, but in fact using both yields better results than the sum of them individually. The high agreement of on automobiles (IoU > 0.40), frogs (IoU > 0.584), and ships (IoU > 0.519) happens when both methods converge on obvious large structural regions, such as a vehicle chassis or a large outline. Moderate agreement (IoU = 0.30 – 0.40) happens when there is partial overlap where the Grad-CAM and Lime methods focus on slightly different aspects of the image. Low agreement on deer (IoU = 0.298) reveals that Grad-CAM and LIME have very different areas and patterns they focus on when classifying a image. Grad-CAM still focuses on the head and torso of the deer, while LIME focuses on the background and antlers. The deciding factors that distinguishes a deer from horses to give one example is the combination of these features, that none of the methods find alone. Grad-CAM mostly focus on broad gradient based activations like diffuse fur patterns. While LIME focus on discrete sensitive perturbation in pixels, that we might find in facial details. Thus, practitioners wanting explainable classification should use both methods together to increase the robustness of the diagnostics. From our results there is clearly a correlation between high IoU and the confidence we should have that the answer is correct. In the case of frogs, this animal doesn't get the same issues that fur animals get. Due to both methods seeing the clear shape of the animal. The fur doesn't obscure and create texture bias and therefore we get a higher IoU and the algorithm confidently classifies at a higher percent these images as frogs.

3) *Strengths*: The combined use of the Grad-CAM heatmap and LIME-green boundaries creates a highly effective visual explanation for assessing whether the result is agreeable. These methods works great visually separated to view different perspectives of how a image was classified or together, you can view both in the results. The Grad-CAM method provides heatmaps, that show broad class discriminative shapes and areas. While the LIME method provides sensitive perturbations

in details. These together work in collaboration and function as a reserve method for each other. The accuracy is high for this model, while maintaining a reasonable training time on a standard working computer (MacBook Pro M1). This model with the XAI methods are easy to implement on a standard working computer and runs relatively fast, and therefore this is available for many applications where processing power and time have limitations.

4) *Implications:* Our findings demonstrates that there is practical value using a combined Grad-CAM and LIME model to achieve explainable applicable computer vision. This can be used in critical and systems like autonomous driving, where the model must distinguish the cars, pedestrians, signs, animals and edges of the road from each other. And if later there where to be a accident, the records from the event should offer explainability so that we can eliminate the specific accident from happening again. Also developers can find trends where the IoU is low, such as with cats, dogs and deer in our case. And preemptively know which problems the computer vision must resolve before it is safe to employ in the real world.

IoU is a great tool to identify uncertainty for reliable explanations. High IoU indicates a overlap in gradient and perturbation based methods, this indicates that regions highlighted are identified correctly and boosts the trustworthiness of the classification. A moderate IoU indicates that there is overlapping but still some variation between the methods, thus creating increased disagreement and less trust in the classification. Because these cases have a higher error rate. And low IoU might indicate a red flag, that there is a great uncertainty in the explanation and that this should activate possible errors in the pipeline, trigger human reviews, retrain the model or other solutions when applied to real-world problems.

We have made some suggestions for a few example scenarios we believe these methods to yield results. These utilize one or more concepts discussed in this article. They are general recommendations provided for others looking to solve problems with explainable AI in computer vision. In the table II below, we have a guide for practitioners in selecting appropriate XAI methods, we propose the following application guidelines.

5) *Limitations:* Given the datasets available, and that we are running this on a standard mac computer, we had to use a small sample size. This might be the largest limitation from this specific implementation. If we had more computing power we would like to test the same implementation just with higher resolution images and a larger dataset. CIFAR-10 (32×32 pixels), upsampled to (224×224) via bilinear interpolation. Testing more categories and more images per category, would increase the results and provide a better base for analysis. When you are utilizing low resolution this doesn't correctly reflect the world, if this was to be used in a real world scenario the algorithm needs to be tested on higher resolution to actually distinguish different shapes and objects from each other and to increase the level of details. Binary mask oversimplifies the nuanced output of Grad-CAM, that can

TABLE II
XAI METHOD SELECTION GUIDELINES

Scenario	Recommended Method	Rationale
Global structure focus (e.g., vehicles, ships)	Grad-CAM	Provides smooth heatmaps highlighting broad, class-discriminative regions
Local texture details (e.g., animals with fur)	LIME	Identifies discrete superpixels sensitive to perturbations in fine-grained features
High-uncertainty classes (e.g., deer, birds)	Both + IoU	Complementary views build consensus; low IoU flags unreliable decisions
Resource-constrained environments	Grad-CAM	Faster computation (no perturbations) suits real-time analysis
Interpretability for non-experts	LIME	Superpixel boundaries offer intuitive, segment-based explanations

produce continuous heatmaps and LIMEs ranked superpixels. Empirical tests showed threshold variations (e.g., 60th–80th percentile or $K=5-15$) altered IoU by up to ± 0.08 , affecting agreement interpretations. Also restricting the analysis to a single ResNet-50 architecture gives limited insights into how Grad-CAM and LIME perform across different models. Such as vision transformers, where attention mechanisms might yield different spatial patterns. These factors constrain the generalization of our findings beyond this controlled setup. Other limitations to this approach are not testing other explainable AI methods such as SHAP (SHapley Additive exPlanations) [14], and comparing our results with an alternative explainable method. Therefore we cannot be certain that other methods would work better in some scenarios or provide other productive insights into the reasoning behind the AI's classification. But for the scope of this project, given the limits in time, computing power and goals we did not try methods such as this.

6) *Possible Improvements:* The main improvement to this implementation, is using higher resolution images. Higher resolution images could possibly give clearer boundaries separating regions and make the heatmaps more informative, by allowing more clear highlighting of informative features.

Another possible improvement is a larger set of images and classes. Checking to see if there are other classes of images that perform worse, or possibly better with this implementation could yield more insight into why certain classes are misclassified with others. Examples like horses and deer being confused with each other and possible examples of other could reveal more insight into biases the model has and where it is easily confused.

In order to validate if the model is generalizing it would be possible to employ random noise to muddle the dataset. We could attempt to see how robust the model is and how noise can throw off the classification.

It is also possible to use other architectures, such as Vision Transformers. This could test the XAI beyond CNNs, this might change the agreement patterns or provide insights into

how different models put emphasis on different parts of images to arrive at their classification. Transformers also offer the possibility of using attention mechanisms for XAI as another possible algorithm to evaluate which parts are focused on by transformers.

In the same spirit it is also possible to use different XAI methods, like SHAP or integrated gradients, as alternative methods. Again a IoU could be computed to allow assessment of effectiveness. It would allow a much larger comparison between multiple algorithms.

Real-time applications are an important field of computer vision. For certain real-time applications explainability is also very important, such as self-driving cars. As such it would be interesting to try to employ XAI in combination with real-time applications, to see where the limitations of such systems lay and how the trade-off of results versus computational time affect the results.

VI. CONCLUSION

Our implementation fine-tuned a pretrained ResNet model on the CIFAR-10 dataset using class-level monitoring to prevent overfitting. The trained model achieved a mean accuracy of 93.3 %, with per-class accuracy ranging from 86-97%, demonstrating robust performance across all categories. Comparative XAI analysis using 2 samples per class shows a mean IoU of 0.364 between Grad-CAM and LIME.

Class-level analysis showed variation in explanation consistency. Ships and trucks achieved the highest IoU scores (> 0.45), indicating stable discriminative features, while deer exhibited the lowest agreement IoU < 0.30 , suggesting reliance on distributed or contextual cues. This variation shows that some object categories produce more consistent explanations across methods than others.

The spatial IoU metric provides a quantitative benchmark for comparing XAI techniques, bridging the gap between gradient-based and perturbation-based approaches. Our framework demonstrates that combining complementary explainable methods enable cross-validation of model decisions. The approach is computationally accessible, making it suitable for both education and deployment scenarios.

However, several considerations need to be accounted for. The moderate IoU score (0.364) reflects the complementary nature of the two methods. Grad-CAM highlights regions with high gradient flow, while LIME identifies superpixels. This divergence is expected and valuable for robustness validation, as perfect agreement might indicate redundancy. However, low IoU on certain classes (e.g., deer) suggests feature instability and requires further investigation. Additionally, CIFAR-10's low resolution (32x32 pixels upsampled to 224x224) may limit spatial granularity. Validation on native high-resolution datasets is necessary to better assess generalization.

Our results shows a moderate mean agreement, and for specific categories both high and low agreement depending on the specific category. We can conclude that these two methods are complementary in nature and not redundant. Grad-CAM

and LIME provide spatially consistent explanations, which are varied through the different classes of images.

Further work would look at: (1) high-resolution dataset validation to assess whether explanation consistency improves with richer spatial information, (2) analysis of visual features (texture, shape, color) that drive consistency across methods, and (3) extension to other XAI techniques (e.g., Integrated Gradients, SHAP).

VII. DECLARATION OF AI USAGES

We have used AI as a debugging, and coding tool for the technical parts of this assignment. We have also used AI to get feedback on our report, as a way to get continuous criticism and suggestions. The report is written by us.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2012, pp. 1097–1105.
- [2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2015.
- [3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.
- [4] A. Krizhevsky, V. Nair, and G. Hinton, "The CIFAR-10 dataset," Available: <https://www.cs.toronto.edu/~kriz/cifar.html>, 2012, accessed: Nov. 12, 2025.
- [5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [6] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in Neural Information Processing Systems*, 2014, pp. 3320–3328.
- [7] H. C. Yoon and L. P. Lin, "Brain Tumor Classification in MRI: Insights From LIME and Grad-CAM Explainable AI Techniques," *IEEE Access*, vol. 13, pp. 154 172–154 202, 2025.
- [8] Z. Cheng, Y. Wu, Y. Li, L. Cai, and B. Ihnaini, "A comprehensive review of explainable artificial intelligence (XAI) in computer vision," *Sensors*, vol. 25, no. 13, p. 4166, 2025.
- [9] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.
- [10] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2016, pp. 1135–1144.
- [11] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.
- [12] J. J. Bird and A. Lotfi, "CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images," *IEEE Access*, vol. 12, pp. 15 642–15 650, 2024.
- [13] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, and W. Brendel, "ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness," *arXiv preprint arXiv:1811.12231*, 2018.
- [14] F. P. Herrera, L. Rokach, N. Díaz-Rodríguez, and B. Shapira, "Shapley additive explanation," *Computers in Biology and Medicine*, 2023.

APPENDIX

GithubRepo

November 28, 2025

1 PART 1: SETUP & PREPROCESSING

1.0.1 Import Required Libraries

```
[8]: """
    Explainable Image Classification on CIFAR-10

    This notebook implements the complete pipeline for
    1. Model Selection and Training (ResNet-50)
    2. Data Preprocessing (CIFAR-10)
    3. Fine-Tuning with Transfer Learning
    4. XAI Explanations (Grad-CAM and LIME)
    """

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader, random_split
import numpy as np
import matplotlib.pyplot as plt
from tqdm.notebook import tqdm
import time

# Set random seeds for reproducibility
torch.manual_seed(42)
np.random.seed(42)

print("Libraries imported successfully!")
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"MPS (Apple Silicon GPU) available: {torch.backends.mps.is_available()}")
print(f"MPS built: {torch.backends.mps.is_built()}")
```

```
Libraries imported successfully!
PyTorch version: 2.5.1
CUDA available: False
MPS (Apple Silicon GPU) available: True
MPS built: True
```

1.0.2 Device Setup

```
[9]: def setup_device():
    """
    Configure the computation device for optimal performance.

    Device Priority:
    1. MPS (Metal Performance Shaders) - Apple Silicon GPU (M1/M2/M3)
    2. CUDA - NVIDIA GPU
    3. CPU - Fallback option

    Returns:
        torch.device: The device to be used for tensor operations
    """
    # Check for Apple Silicon GPU (MPS)
    if torch.backends.mps.is_available() and torch.backends.mps.is_built():
        device = torch.device("mps")
        print("\n" + "="*80)
        print("DEVICE CONFIGURATION")
        print("="*80)
        print("Using: Apple Silicon GPU (Metal Performance Shaders)")
        print("  M1/M2/M3/M4 GPU acceleration enabled")
        print("  Unified memory architecture optimized")
    # Check for NVIDIA GPU (CUDA)
    elif torch.cuda.is_available():
        device = torch.device("cuda")
        print("\n" + "="*80)
        print("DEVICE CONFIGURATION")
        print("="*80)
        print(f"Using: NVIDIA GPU - {torch.cuda.get_device_name(0)}")
        print(f"GPU Memory: {torch.cuda.get_device_properties(0).total_memory / (1024**3):.2f} GB")
        print("  CUDA acceleration enabled")
    # Fallback to CPU
    else:
        device = torch.device("cpu")
        print("\n" + "="*80)
        print("DEVICE CONFIGURATION")
        print("="*80)
        print("Using: CPU")
        print("  GPU acceleration not available - training will be slower")
        print("Consider using Google Colab for free GPU access")

    print("="*80)
    return device

# Initialize device
```

```
device = setup_device()
```

```
=====
DEVICE CONFIGURATION
=====
```

```
Using: Apple Silicon GPU (Metal Performance Shaders)
M1/M2/M3/M4 GPU acceleration enabled
Unified memory architecture optimized
=====
```

1.0.3 Dataset Loadin and Preprocessing

```
[17]: def load_cifar10_data(batch_size=64, num_workers=0):
    """
    Load and preprocess CIFAR-10 dataset for ResNet-50.

    Dataset: 60,000 images (32×32) in 10 classes
    Preprocessing: Resize to 224×224, ImageNet normalization
    Split: 40k train / 10k validation / 10k test

    Args:
        batch_size (int): Samples per batch (64 optimal for M1)
        num_workers (int): Data loading threads (0 for MPS)

    Returns:
        tuple: (train_loader, val_loader, test_loader, class_names)
    """
    # ImageNet normalization constants
    # RGB channel means: Red=0.485, Green=0.456, Blue=0.406
    mean = [0.485, 0.456, 0.406]
    # RGB channel standard deviations: Red=0.229, Green=0.224, Blue=0.225
    std = [0.229, 0.224, 0.225]

    # Training augmentation pipeline (increases dataset diversity, prevents
    ↪ overfitting)
    train_transform = transforms.Compose([
        transforms.Resize(224), # Resize to 224×224
        transforms.RandomHorizontalFlip(p=0.5), # Flip 50% of images
    ↪ horizontally
        transforms.RandomCrop(224, padding=4), # Random crop with 4-pixel
    ↪ padding for slight variations
        transforms.ColorJitter(brightness=0.2, contrast=0.2), # Random
    ↪ brightness/contrast adjustment (±20%)
        transforms.ToTensor(), # Convert to tensor
        transforms.Normalize(mean, std) # Normalize using ImageNet statistics
    ↪ (required for pretrained models)
```

```

])
# Validation/test transform (no augmentation, only preprocessing)
test_transform = transforms.Compose([
    transforms.Resize(224), # Resize to 224x224
    transforms.ToTensor(), # Convert to tensor [0, 1]
    transforms.Normalize(mean, std) # Normalize using ImageNet statistics
])
# Download CIFAR-10
print("Loading CIFAR-10 dataset...")
train_val_dataset = datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=train_transform
)

test_dataset = datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=test_transform
)

# Split train into train/val (80/20)
train_size = int(0.8*len(train_val_dataset)) # 40,000 training samples
val_size = len(train_val_dataset) - train_size # 10,000 validation samples
# Perform random split with fixed seed for reproducibility
train_dataset, val_dataset = random_split(
    train_val_dataset, # Dataset to split (50,000 images)
    [train_size, val_size], # Split sizes [40,000, 10,000]
    generator=torch.Generator().manual_seed(42) # Fixed seed for
    ↪reproducible splits
)

# Apply test transform to validation set (remove augmentation)
# Validation should not have data augmentation like training set
val_dataset.dataset = datasets.CIFAR10(
    root='./data',
    train=True, # Use training set pool
    download=False,
    transform=test_transform # Apply test transform (no augmentation)
)

# Create training data loader (shuffled batches for training)
train_loader = DataLoader(
    train_dataset, # Training dataset (40,000 samples)
    batch_size=batch_size, # Number of samples per batch (64)

```

```

        shuffle=True, # Shuffle data every epoch (prevents learning order)
        num_workers=num_workers, # Number of subprocesses for data loading
        pin_memory=False # Not needed for MPS
    )
    # Create validation data loader (no shuffling needed)
    val_loader = DataLoader(
        val_dataset, # Validation dataset (10,000 samples)
        batch_size=batch_size, # Batch size (64)
        shuffle=False, # No shuffling for validation (order doesn't matter)
        num_workers=num_workers, # Data loading threads (0)
        pin_memory=False # Not needed for MPS
    )
    # Create test data loader (no shuffling needed)
    test_loader = DataLoader(
        test_dataset, # Test dataset (10,000 samples)
        batch_size=batch_size, # Batch size (64)
        shuffle=False, # No shuffling for test set
        num_workers=num_workers, # Data loading threads (0)
        pin_memory=False # Not needed for MPS
    )

    # Class names
    classes = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

    # Print statistics
    print(f"\nDataset Statistics:")
    print(f"  Training samples: {len(train_dataset):,}")
    print(f"  Validation samples: {len(val_dataset):,}")
    print(f"  Test samples: {len(test_dataset):,}")
    print(f"  Number of classes: {len(classes)}")
    print(f"  Batch size: {batch_size}")
    print(f"  Image size: 224×224×3")
    print(f"  Normalization: mean={mean}, std={std}")

    return train_loader, val_loader, test_loader, classes

# Load CIFAR-10 data
train_loader, val_loader, test_loader, class_names = \
    load_cifar10_data(batch_size=64, num_workers=0)

```

Loading CIFAR-10 dataset...

Files already downloaded and verified

Files already downloaded and verified

Dataset Statistics:

Training samples: 40,000

Validation samples: 10,000

Test samples: 10,000

Number of classes: 10
Batch size: 64
Image size: 224×224×3
Normalization: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]

1.0.4 Visualize Sample Images

```
[4]: def visualize_cifar10_samples(loader, classes, num_samples=10):  
    """  
    Display random CIFAR-10 samples after preprocessing.  
  
    Args:  
        loader (DataLoader): Data loader  
        classes (list): Class names  
        num_samples (int): Number of samples to display  
    """  
    # Get batch  
    dataiter = iter(loader)  
    images, labels = next(dataiter)  
  
    # Denormalization constants  
    mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)  
    std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)  
  
    # Create subplots  
    fig, axes = plt.subplots(2, 5, figsize=(15, 6))  
    fig.suptitle('Sample CIFAR-10 Images (224×224)', fontsize=16)  
  
    for idx in range(num_samples):  
        ax = axes[idx // 5, idx % 5]  
  
        # Denormalize  
        img = images[idx] * std + mean  
        img = torch.clamp(img, 0, 1)  
        img = img.permute(1, 2, 0).numpy()  
  
        # Display  
        ax.imshow(img)  
        ax.set_title(f'{classes[labels[idx]]}')  
        ax.axis('off')  
  
    plt.tight_layout()  
    plt.show()  
  
    # Visualize samples  
    print("\nVisualizing training samples:")  
    visualize_cifar10_samples(train_loader, class_names)
```

Visualizing training samples:



2 PART 2: MODEL ARCHITECTURE

2.0.1 Load Pretrained ResNet-50

```
[10]: def load_pretrained_resnet50(num_classes=10, pretrained=True, use_dropout=True):  
    """  
    Load ResNet-50 with optional DROPOUT for regularization.  
  
    Args:  
        num_classes (int): Output classes (10 for CIFAR-10)  
        pretrained (bool): Load ImageNet weights  
        use_dropout (bool): Add dropout layer before final FC  
  
    Returns:  
        nn.Module: Modified ResNet-50  
    """  
    print("Loading ResNet-50 model...")  
  
    # Load pretrained model  
    if pretrained:  
        model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)  
        print("    Loaded ImageNet pretrained weights")  
    else:  
        model = models.resnet50(weights=None)  
        print("    Initialized with random weights")  
  
    # Replace final FC layer with DROPOUT  
    num_features = model.fc.in_features
```

```

if use_dropout:
    # Add Dropout before final layer
    model.fc = nn.Sequential(
        nn.Dropout(p=0.5), # Drop 50% of neurons randomly
        nn.Linear(num_features, num_classes)
    )
    print(f"\n    Replaced FC layer with Dropout: {num_features} →
↳Dropout(0.5) → {num_classes} classes")
else:
    # Original (no dropout)
    model.fc = nn.Linear(num_features, num_classes)
    print(f"\n    Replaced FC layer: {num_features} → {num_classes}
↳classes")

# Calculate parameters
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.
↳requires_grad)

print(f"\nParameters:")
print(f"    Total: {total_params:,}")
print(f"    Trainable: {trainable_params:,}")
print(f"    Dropout enabled: {use_dropout}")

return model

# Usage: Load model with dropout
model = load_pretrained_resnet50(num_classes=10, pretrained=True,
↳use_dropout=True)
model = model.to(device)

```

Loading ResNet-50 model...

Loaded ImageNet pretrained weights

Replaced FC layer with Dropout: 2048 → Dropout(0.5) → 10 classes

Parameters:

Total: 23,528,522

Trainable: 23,528,522

Dropout enabled: True

2.0.2 Training Configuration

```
[ ]: def configure_training(model, learning_rate=0.0005):  
    """  
    Configure loss function, optimizer, and learning rate scheduler.  
  
    Strategy: Fine-tune all layers with Adam optimizer  
    Loss: CrossEntropyLoss for multi-class classification  
    Scheduler: ReduceLROnPlateau (reduce LR when validation loss plateaus)  
  
    Args:  
        model (nn.Module): The model to train  
        learning_rate (float): Initial learning rate  
  
    Returns:  
        tuple: (criterion, optimizer, scheduler)  
    """  
    # Define loss function for multi-class classification  
    # CrossEntropyLoss combines LogSoftmax and NLLLoss  
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1) # Prevent  
↪ overconfidence  
  
    # Configure Adam optimizer (adaptive learning rate optimization)  
    # Optimizes all model parameters (full fine-tuning)  
    optimizer = optim.Adam(  
        # All model parameters will be updated  
        model.parameters(),  
        # Initial learning rate  
        lr=learning_rate,  
        # Weight decay for L2 regularization (prevents overfitting)  
        weight_decay=5e-4  
    )  
  
    # Learning rate scheduler (reduces LR when validation loss plateaus)  
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(  
        # Optimizer to adjust  
        optimizer,  
        # Minimize validation loss  
        mode='min',  
        # Reduce LR by factor of 0.1 when plateau detected  
        factor=0.1,  
        # Wait 3 epochs before reducing LR  
        patience=3,  
        # Verbose output when LR changes  
        verbose=True  
    )
```

```

print("\n" + "="*80)
print("TRAINING CONFIGURATION")
print("="*80)
print(f"Loss Function: CrossEntropyLoss")
print(f"Optimizer: Adam")
print(f"Learning Rate: {learning_rate}")
print(f"Weight Decay: 1e-4")
print(f"LR Scheduler: ReduceLROnPlateau (factor=0.1, patience=3)")
print(f"Training Strategy: Full fine-tuning (all layers trainable)")
print("="*80)

return criterion, optimizer, scheduler

# Configure training components
criterion, optimizer, scheduler = configure_training(model, learning_rate=0.
↪0005)

```

3 PART 3: TRAINING & FINE-TUNING

3.0.1 Training Functions

```

[7]: def train_one_epoch(model, train_loader, criterion, optimizer, device,
↪clip_grad=1.0):
    """
    Train the model for one epoch.

    Process:
    1. Set model to training mode
    2. Iterate through batches
    3. Forward pass → compute loss → backward pass → update weights
    4. Track loss and accuracy

    Args:
        model (nn.Module): The model to train
        train_loader (DataLoader): Training data loader
        criterion: Loss function
        optimizer: Optimizer
        device: Device to use (MPS/CUDA/CPU)

    Returns:
        tuple: (average_loss, accuracy)
    """
    # Set model to training mode
    model.train()
    running_loss = 0.0
    correct = 0 # Correct predictions counter
    total = 0

```

```

# Create progress bar for training batches
train_pbar = tqdm(train_loader, desc='Training', leave=False)

# Iterate through training batches
for inputs, labels in train_pbar:
    # Move inputs for device
    inputs = inputs.to(device)
    # Move labels to device
    labels = labels.to(device)

    # Zero the parameter gradients (clear previous gradients)
    optimizer.zero_grad()
    # Forward pass: compute model predictions
    outputs = model(inputs)
    # Compute loss between predictions and true labels
    loss = criterion(outputs, labels)
    # Backward pass: compute gradients
    loss.backward()
    # GRADIENT CLIPPING (prevents exploding gradients)
    if clip_grad > 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(),
↪max_norm=clip_grad)
    # Update model parameters using computed gradients
    optimizer.step()

    # Accumulate loss
    running_loss += loss.item() * inputs.size(0)

    # Calculate accuracy
    # Get predicted class (highest probability)
    _, predicted = torch.max(outputs, 1)
    # Count total samples in batch
    total += labels.size(0)
    # Count correct predictions
    correct += (predicted == labels).sum().item()

    # Update progress bar with current metrics
    train_pbar.set_postfix({
        'loss': f'{loss.item():.4f}',
        'acc': f'{100.0 * correct / total:.2f}%'
    })

    # Calculate epoch metric
    # Average loss across all samples
    epoch_loss = running_loss / total
    # Accuracy as percentage
    epoch_acc = 100.0 * correct / total

```

```

return epoch_loss, epoch_acc

def validate(model, val_loader, criterion, device):
    """
    Validate the model on validation set.

    Process:
    1. Set model to evaluation mode
    2. Disable gradient computation
    3. Iterate through batches
    4. Compute loss and accuracy

    Args:
        model (nn.Module): The model to validate
        val_loader (DataLoader): Validation data loader
        criterion: Loss function
        device: Device to use

    Returns:
        tuple: (average_loss, accuracy)
    """
    # Set model to evaluation mode (disables dropout, uses batch norm running_
    ↪ stats)
    model.eval()

    # Initialize metrics
    running_loss = 0.0
    correct = 0
    total = 0

    # Disable gradient computation (saves memory and speeds up validation)
    with torch.no_grad():
        # Create progress bar for validation batches
        val_pbar = tqdm(val_loader, desc='Validation', leave=False)

        # Iterate through validation batches
        for inputs, labels in val_pbar:
            # Move data to device
            inputs = inputs.to(device)
            labels = labels.to(device)

            # Forward pass only (no backward pass)
            outputs = model(inputs)
            # Compute loss
            loss = criterion(outputs, labels)

            # Accumulate loss (multiply by batch size for weighted average)

```

```

        running_loss += loss.item() * inputs.size(0)

        # Calculate accuracy
        # Get predicted class (highest probability)
        _, predicted = torch.max(outputs, 1)
        # Count total samples
        total += labels.size(0)
        # Count correct predictions
        correct += (predicted == labels).sum().item()

        # Update progress bar with current metrics
        val_pbar.set_postfix({
            'loss': f'{loss.item():.4f}',
            'acc': f'{100.0 * correct / total:.2f}%'
        })

    # Calculate validation metrics
    val_loss = running_loss / total
    val_acc = 100.0 * correct / total

    return val_loss, val_acc

```

3.0.2 Main Training Loop

```

[8]: def train_model_with_class_monitoring(model, train_loader, val_loader,
      ↪ criterion,
                                     optimizer, scheduler, device, class_names,
                                     num_epochs=20, save_path='best_model.
      ↪ pth'):
    """
    ENHANCED: Train model with per-class accuracy monitoring to prevent
    ↪ degradation.
    OPTIMIZED: Uses vectorized operations for 10x faster per-class tracking.

    Key Features:
    1. Tracks overall validation accuracy
    2. Monitors per-class accuracy to catch class-specific failures
    3. Only saves model if BOTH conditions met:
       - Overall accuracy improves
       - No class degrades by more than 5%
    4. Early stopping based on overall accuracy plateau
    5. Detailed per-class reporting every epoch
    6. OPTIMIZED: Vectorized per-class tracking (no slow Python loops)

    Args:
        model (nn.Module): Model to train
        train_loader (DataLoader): Training data

```

```

    val_loader (DataLoader): Validation data
    criterion: Loss function
    optimizer: Optimizer
    scheduler: Learning rate scheduler
    device: Device to use (MPS/CUDA/CPU)
    class_names (list): List of class names for CIFAR-10
    num_epochs (int): Maximum training epochs (default: 30)
    save_path (str): Path to save best model weights

Returns:
    dict: Training history including per-class metrics
"""

# =====
# STEP 1: INITIALIZE TRACKING VARIABLES
# =====

# Initialize history dictionary to store training metrics over time
history = {
    'train_loss': [],      # Training loss per epoch
    'train_acc': [],      # Training accuracy per epoch
    'val_loss': [],       # Validation loss per epoch
    'val_acc': [],        # Validation accuracy per epoch
    'per_class_acc': []   # Per-class accuracy per epoch (list of dicts)
}

# Track best overall validation accuracy (for model saving)
best_val_acc = 0.0

# Track best minimum class accuracy (worst-performing class)
best_min_class_acc = 0.0

# Early stopping counter
epochs_without_improvement = 0

# Number of epochs to wait before triggering early stopping
patience = 5

# Record training start time
start_time = time.time()

# =====
# STEP 2: PRINT TRAINING CONFIGURATION
# =====

print("\n" + "="*80)
print("STARTING ENHANCED TRAINING WITH PER-CLASS MONITORING (OPTIMIZED)")

```

```

print("="*80)
print(f"Max Epochs: {num_epochs}")
print(f"Early Stopping Patience: {patience}")
print(f"Classes Monitored: {len(class_names)}")
print(f"Model Save Path: {save_path}")
print(f"Optimization: Vectorized per-class tracking")
print("="*80)

# =====
# STEP 3: MAIN TRAINING LOOP
# =====

for epoch in range(num_epochs):
    # Print epoch header
    print(f"\nEpoch {epoch+1}/{num_epochs}")
    print("-" * 80)

    # -----
    # STEP 3.1: TRAINING PHASE
    # -----

    # Train for one epoch
    train_loss, train_acc = train_one_epoch(
        model, train_loader, criterion, optimizer, device
    )

    # -----
    # STEP 3.2: VALIDATION PHASE (OPTIMIZED PER-CLASS TRACKING)
    # -----

    # Set model to evaluation mode
    model.eval()

    # Initialize validation metrics
    running_loss = 0.0
    correct = 0
    total = 0

    # Initialize per-class tracking dictionaries
    class_correct = {i: 0 for i in range(len(class_names))}
    class_total = {i: 0 for i in range(len(class_names))}

    # Disable gradient computation
    with torch.no_grad():
        # Create progress bar
        val_pbar = tqdm(val_loader, desc='Validation', leave=False)

```

```

# Iterate through validation batches
for inputs, labels in val_pbar:
    # Move data to device
    inputs = inputs.to(device)
    labels = labels.to(device)

    # Forward pass
    outputs = model(inputs)

    # Compute loss
    loss = criterion(outputs, labels)

    # Accumulate loss
    running_loss += loss.item() * inputs.size(0)

    # Get predicted class
    _, predicted = torch.max(outputs, 1)

    # Update overall accuracy counters
    total += labels.size(0)
    correct += (predicted == labels).sum().item()

    # =====
    # OPTIMIZED: Vectorized per-class tracking (10x faster!)
    # =====
    # Move to CPU ONCE (instead of 64 times in a loop)
    labels_cpu = labels.cpu().numpy()
    predicted_cpu = predicted.cpu().numpy()

    # Vectorized counting (NumPy operations, very fast)
    for class_idx in range(len(class_names)):
        # Create boolean mask for current class
        class_mask = (labels_cpu == class_idx)
        # Count samples for this class
        class_total[class_idx] += class_mask.sum()
        # Count correct predictions for this class
        class_correct[class_idx] += ((predicted_cpu == labels_cpu)
↪ & class_mask).sum()
    # =====

    # Update progress bar
    val_pbar.set_postfix({
        'loss': f'{loss.item():.4f}',
        'acc': f'{100.0 * correct / total:.2f}%'
    })

# -----

```

```

# STEP 3.3: CALCULATE EPOCH METRICS
# -----

# Calculate overall validation loss
val_loss = running_loss / total

# Calculate overall validation accuracy
val_acc = 100.0 * correct / total

# Calculate per-class accuracy
per_class_acc = {}
for i in range(len(class_names)):
    if class_total[i] > 0:
        per_class_acc[class_names[i]] = 100.0 * class_correct[i] / ↵
class_total[i]
    else:
        per_class_acc[class_names[i]] = 0.0

# Find minimum class accuracy (worst-performing class)
min_class_acc = min(per_class_acc.values())

# Calculate average class accuracy
avg_class_acc = np.mean(list(per_class_acc.values()))

# -----
# STEP 3.4: UPDATE LEARNING RATE SCHEDULER
# -----

# ReduceLRonPlateau: reduce LR if validation loss plateaus
scheduler.step(val_loss)

# -----
# STEP 3.5: STORE METRICS IN HISTORY
# -----

history['train_loss'].append(train_loss)
history['train_acc'].append(train_acc)
history['val_loss'].append(val_loss)
history['val_acc'].append(val_acc)
history['per_class_acc'].append(per_class_acc)

# -----
# STEP 3.6: PRINT EPOCH RESULTS
# -----

print(f"\nResults:")
print(f"  Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}%")

```

```

print(f" Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}%")
print(f" Min Class Acc: {min_class_acc:.2f}% | Avg Class Acc:␣
↪{avg_class_acc:.2f}%")
print(f" Current LR: {optimizer.param_groups[0]['lr']:.6f}")

# Print per-class accuracy (sorted from worst to best)
print(f"\n Per-Class Accuracy:")
for class_name, acc in sorted(per_class_acc.items(), key=lambda x:␣
↪x[1]):
    # Add emoji indicator based on performance
    if acc > 70:
        emoji = " 🏆 "
    elif acc > 60:
        emoji = " 🥈 "
    else:
        emoji = " 🥉 "
    print(f" {emoji} {class_name:12s}: {acc:.2f}%")

# -----
# STEP 3.7: MODEL SAVING LOGIC (ENHANCED)
# -----

# Save model only if BOTH conditions are met
save_condition = (
    val_acc > best_val_acc and
    min_class_acc >= best_min_class_acc * 0.95
)

if save_condition:
    # Update best metrics
    best_val_acc = val_acc
    best_min_class_acc = min_class_acc

    # Reset early stopping counter
    epochs_without_improvement = 0

    # Save model
    torch.save(model.state_dict(), save_path)

    # Print success message
    print(f"\n NEW BEST MODEL! Val Acc: {val_acc:.2f}% | Min Class:␣
↪{min_class_acc:.2f}%")
    else:
        # Increment early stopping counter
        epochs_without_improvement += 1

        # Check if overall improved but class performance degraded

```

```

        if val_acc > best_val_acc and min_class_acc < best_min_class_acc * 0.95:
            print(f"\n    Overall improved but class performance degraded,
(not saved)")
            print(f"    Worst class dropped from {best_min_class_acc:.2f}%
to {min_class_acc:.2f}%")

            # Print early stopping status
            print(f"    No improvement for {epochs_without_improvement}/
{patience} epochs")

# -----
# STEP 3.8: CHECK EARLY STOPPING CONDITION
# -----

if epochs_without_improvement >= patience:
    # Early stopping triggered
    print(f"\n    EARLY STOPPING triggered after {epoch+1} epochs")
    print(f"    Best Val Acc: {best_val_acc:.2f}%")
    print(f"    Best Min Class Acc: {best_min_class_acc:.2f}%")
    break

# =====
# STEP 4: TRAINING COMPLETED - PRINT SUMMARY
# =====

# Calculate total training time
total_time = time.time() - start_time

print("\n" + "="*80)
print("TRAINING COMPLETED")
print("="*80)
print(f"Total Training Time: {total_time/3600:.2f} hours")
print(f"Total Epochs Run: {len(history['train_loss'])}")
print(f"Best Validation Accuracy: {best_val_acc:.2f}%")
print(f"Best Minimum Class Accuracy: {best_min_class_acc:.2f}%")
print(f"Best model saved to: {save_path}")
print("="*80)

# Return training history
return history

# =====
# USAGE: TRAIN THE MODEL (SAME FUNCTION CALL AS BEFORE)
# =====

```

```

# Train the model - NO CHANGES NEEDED TO YOUR EXISTING CODE!
history = train_model_with_class_monitoring(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    class_names=class_names,
    num_epochs=20,
    save_path='resnet50_cifar10_anti_overfitting.pth'
)

```

```

=====
STARTING ENHANCED TRAINING WITH PER-CLASS MONITORING (OPTIMIZED)
=====

```

```

Max Epochs: 20
Early Stopping Patience: 5
Classes Monitored: 10
Model Save Path: resnet50_cifar10_anti_overfitting.pth
Optimization: Vectorized per-class tracking
=====

```

```

Epoch 1/20
-----

```

```

Training:   0%|          | 0/625 [00:00<?, ?it/s]
Validation: 0%|          | 0/157 [00:00<?, ?it/s]

```

```

Results:

```

```

  Train Loss: 1.1110 | Train Acc: 74.70%
  Val Loss:   1.1805 | Val Acc:   71.17%
  Min Class Acc: 60.19% | Avg Class Acc: 71.06%
  Current LR: 0.000500

```

```

Per-Class Accuracy:

```

```

  cat       : 60.19%
  truck     : 60.66%
  airplane  : 61.36%
  deer      : 63.34%
  automobile : 66.53%
  horse     : 68.12%
  bird      : 74.16%
  dog       : 74.61%
  frog      : 83.32%

```

ship : 98.30%

NEW BEST MODEL! Val Acc: 71.17% | Min Class: 60.19%

Epoch 2/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 1.0145 | Train Acc: 78.97%
Val Loss: 1.0234 | Val Acc: 78.84%
Min Class Acc: 57.94% | Avg Class Acc: 78.70%
Current LR: 0.000500

Per-Class Accuracy:

deer : 57.94%
bird : 59.05%
cat : 64.74%
dog : 76.56%
ship : 81.36%
horse : 81.74%
truck : 82.18%
airplane : 92.19%
frog : 94.93%
automobile : 96.34%

NEW BEST MODEL! Val Acc: 78.84% | Min Class: 57.94%

Epoch 3/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.9504 | Train Acc: 81.70%
Val Loss: 0.9045 | Val Acc: 83.32%
Min Class Acc: 70.63% | Avg Class Acc: 83.29%
Current LR: 0.000500

Per-Class Accuracy:

cat : 70.63%
dog : 76.86%
bird : 80.02%
automobile : 80.99%

frog	: 84.49%
airplane	: 84.58%
deer	: 85.57%
horse	: 88.70%
ship	: 88.88%
truck	: 92.19%

NEW BEST MODEL! Val Acc: 83.32% | Min Class: 70.63%

Epoch 4/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.9045 | Train Acc: 83.79%
Val Loss: 0.9477 | Val Acc: 81.20%
Min Class Acc: 61.74% | Avg Class Acc: 81.04%
Current LR: 0.000500

Per-Class Accuracy:

cat	: 61.74%
airplane	: 69.68%
deer	: 75.18%
bird	: 77.34%
automobile	: 78.22%
dog	: 78.71%
frog	: 84.59%
ship	: 92.18%
horse	: 94.30%
truck	: 98.50%

No improvement for 1/5 epochs

Epoch 5/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.8764 | Train Acc: 84.95%
Val Loss: 0.9256 | Val Acc: 82.33%
Min Class Acc: 57.62% | Avg Class Acc: 82.32%
Current LR: 0.000500

Per-Class Accuracy:

dog : 57.62%
frog : 68.49%
cat : 75.18%
deer : 77.15%
airplane : 83.45%
truck : 84.98%
bird : 87.87%
horse : 95.17%
ship : 95.89%
automobile : 97.43%
No improvement for 2/5 epochs

Epoch 6/20

Training: 0% | 0/625 [00:00<?, ?it/s]
Validation: 0% | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.8503 | Train Acc: 86.14%
Val Loss: 0.8642 | Val Acc: 85.02%
Min Class Acc: 73.16% | Avg Class Acc: 84.95%
Current LR: 0.000500

Per-Class Accuracy:

bird : 73.16%
cat : 77.77%
airplane : 77.90%
dog : 81.25%
automobile : 82.87%
deer : 84.63%
horse : 89.47%
truck : 91.19%
frog : 95.41%
ship : 95.89%

NEW BEST MODEL! Val Acc: 85.02% | Min Class: 73.16%

Epoch 7/20

Training: 0% | 0/625 [00:00<?, ?it/s]
Validation: 0% | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.8337 | Train Acc: 86.60%
Val Loss: 0.8541 | Val Acc: 85.63%

Min Class Acc: 65.56% | Avg Class Acc: 85.53%
Current LR: 0.000500

Per-Class Accuracy:

cat	: 65.56%
deer	: 75.70%
bird	: 80.72%
automobile	: 83.66%
ship	: 88.28%
frog	: 88.39%
horse	: 89.76%
dog	: 91.41%
airplane	: 95.58%
truck	: 96.20%

Overall improved but class performance degraded (not saved)

Worst class dropped from 73.16% to 65.56%

No improvement for 1/5 epochs

Epoch 8/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.8224 | Train Acc: 87.00%

Val Loss: 0.8852 | Val Acc: 83.86%

Min Class Acc: 40.02% | Avg Class Acc: 83.69%

Current LR: 0.000500

Per-Class Accuracy:

cat	: 40.02%
dog	: 77.05%
bird	: 79.42%
deer	: 81.31%
frog	: 89.76%
truck	: 90.59%
horse	: 93.62%
airplane	: 94.04%
ship	: 94.89%
automobile	: 96.24%

No improvement for 2/5 epochs

Epoch 9/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.8082 | Train Acc: 87.93%
Val Loss: 0.8463 | Val Acc: 85.30%
Min Class Acc: 61.01% | Avg Class Acc: 85.18%
Current LR: 0.000500

Per-Class Accuracy:

cat	: 61.01%
deer	: 78.61%
automobile	: 81.68%
bird	: 81.71%
horse	: 85.99%
dog	: 86.43%
airplane	: 88.59%
truck	: 95.10%
frog	: 96.10%
ship	: 96.59%

Overall improved but class performance degraded (not saved)

Worst class dropped from 73.16% to 61.01%

No improvement for 3/5 epochs

Epoch 10/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.8026 | Train Acc: 87.99%
Val Loss: 0.7795 | Val Acc: 88.73%
Min Class Acc: 74.87% | Avg Class Acc: 88.69%
Current LR: 0.000500

Per-Class Accuracy:

cat	: 74.87%
dog	: 83.59%
bird	: 85.69%
horse	: 86.57%
airplane	: 89.11%
deer	: 89.41%
automobile	: 93.27%
frog	: 93.85%
ship	: 95.19%
truck	: 95.40%

NEW BEST MODEL! Val Acc: 88.73% | Min Class: 74.87%

Epoch 11/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.7975 | Train Acc: 88.11%

Val Loss: 0.8353 | Val Acc: 86.01%

Min Class Acc: 63.28% | Avg Class Acc: 86.07%

Current LR: 0.000500

Per-Class Accuracy:

dog : 63.28%

cat : 80.77%

horse : 80.77%

airplane : 84.07%

bird : 84.99%

frog : 90.83%

deer : 91.38%

automobile : 92.67%

truck : 93.89%

ship : 98.00%

No improvement for 1/5 epochs

Epoch 12/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.7935 | Train Acc: 88.23%

Val Loss: 0.8781 | Val Acc: 84.25%

Min Class Acc: 68.97% | Avg Class Acc: 84.31%

Current LR: 0.000500

Per-Class Accuracy:

truck : 68.97%

dog : 71.58%

horse : 77.87%

frog : 82.24%

cat : 84.28%

deer : 85.57%

airplane : 89.31%
bird : 91.45%
automobile : 95.35%
ship : 96.49%
No improvement for 2/5 epochs

Epoch 13/20

Training: 0%| | 0/625 [00:00<?, ?it/s]
Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.7879 | Train Acc: 88.50%
Val Loss: 0.8242 | Val Acc: 86.50%
Min Class Acc: 67.01% | Avg Class Acc: 86.47%
Current LR: 0.000500

Per-Class Accuracy:

cat : 67.01%
bird : 80.72%
frog : 81.95%
horse : 84.73%
deer : 88.79%
ship : 89.58%
dog : 90.04%
automobile : 90.89%
airplane : 93.83%
truck : 97.20%

No improvement for 3/5 epochs

Epoch 14/20

Training: 0%| | 0/625 [00:00<?, ?it/s]
Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.7851 | Train Acc: 88.44%
Val Loss: 0.8428 | Val Acc: 86.00%
Min Class Acc: 74.47% | Avg Class Acc: 85.95%
Current LR: 0.000050

Per-Class Accuracy:

truck : 74.47%
cat : 77.15%
dog : 78.91%

bird : 80.72%
airplane : 84.17%
frog : 88.29%
deer : 90.03%
automobile : 93.86%
ship : 95.89%
horse : 96.04%
No improvement for 4/5 epochs

Epoch 15/20

Training: 0%| | 0/625 [00:00<?, ?it/s]
Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.6880 | Train Acc: 92.99%
Val Loss: 0.6862 | Val Acc: 92.57%
Min Class Acc: 82.83% | Avg Class Acc: 92.54%
Current LR: 0.000050

Per-Class Accuracy:

cat : 82.83%
dog : 87.11%
bird : 89.96%
airplane : 92.39%
deer : 92.83%
horse : 93.82%
frog : 95.90%
automobile : 96.24%
ship : 96.89%
truck : 97.40%

NEW BEST MODEL! Val Acc: 92.57% | Min Class: 82.83%

Epoch 16/20

Training: 0%| | 0/625 [00:00<?, ?it/s]
Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.6534 | Train Acc: 94.57%
Val Loss: 0.6762 | Val Acc: 93.02%
Min Class Acc: 83.66% | Avg Class Acc: 92.99%
Current LR: 0.000050

Per-Class Accuracy:

cat	: 83.66%
dog	: 88.67%
bird	: 91.15%
deer	: 92.11%
airplane	: 94.35%
horse	: 94.59%
frog	: 94.63%
ship	: 95.79%
automobile	: 97.33%
truck	: 97.60%

NEW BEST MODEL! Val Acc: 93.02% | Min Class: 83.66%

Epoch 17/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.6353 | Train Acc: 95.37%
Val Loss: 0.6728 | Val Acc: 93.18%
Min Class Acc: 85.94% | Avg Class Acc: 93.17%
Current LR: 0.000050

Per-Class Accuracy:

dog	: 85.94%
cat	: 86.35%
bird	: 89.86%
deer	: 93.25%
horse	: 94.40%
airplane	: 94.45%
frog	: 95.22%
ship	: 97.19%
automobile	: 97.33%
truck	: 97.70%

NEW BEST MODEL! Val Acc: 93.18% | Min Class: 85.94%

Epoch 18/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.6287 | Train Acc: 95.60%
Val Loss: 0.6761 | Val Acc: 93.06%
Min Class Acc: 81.35% | Avg Class Acc: 93.06%
Current LR: 0.000050

Per-Class Accuracy:

dog	: 81.35%
cat	: 89.45%
bird	: 90.95%
airplane	: 92.09%
horse	: 94.11%
deer	: 94.18%
truck	: 96.70%
frog	: 97.07%
ship	: 97.29%
automobile	: 97.43%

No improvement for 1/5 epochs

Epoch 19/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0%| | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.6180 | Train Acc: 96.11%
Val Loss: 0.6688 | Val Acc: 93.34%
Min Class Acc: 84.67% | Avg Class Acc: 93.32%
Current LR: 0.000050

Per-Class Accuracy:

dog	: 84.67%
cat	: 87.80%
bird	: 90.56%
deer	: 92.73%
airplane	: 94.14%
frog	: 95.41%
ship	: 96.39%
horse	: 96.91%
automobile	: 97.23%
truck	: 97.40%

NEW BEST MODEL! Val Acc: 93.34% | Min Class: 84.67%

Epoch 20/20

Training: 0%| | 0/625 [00:00<?, ?it/s]

Validation: 0% | 0/157 [00:00<?, ?it/s]

Results:

Train Loss: 0.6109 | Train Acc: 96.45%
Val Loss: 0.6690 | Val Acc: 93.19%
Min Class Acc: 85.83% | Avg Class Acc: 93.17%
Current LR: 0.000050

Per-Class Accuracy:

cat : 85.83%
dog : 88.28%
bird : 89.56%
horse : 92.85%
deer : 93.67%
airplane : 93.83%
frog : 96.29%
automobile : 96.93%
truck : 97.20%
ship : 97.29%

No improvement for 1/5 epochs

=====

TRAINING COMPLETED

=====

Total Training Time: 4.49 hours
Total Epochs Run: 20
Best Validation Accuracy: 93.34%
Best Minimum Class Accuracy: 84.67%
Best model saved to: resnet50_cifar10_anti_overfitting.pth

=====

3.0.3 Visualize Training History

```
[9]: def plot_training_history(history):  
    """  
    Plot training and validation metrics over epochs.  
  
    Creates two subplots:  
    1. Loss curves (training and validation)  
    2. Accuracy curves (training and validation)  
  
    Args:  
        history (dict): Training history with losses and accuracies  
    """  
    # Create figure with 2 subplots (1 row, 2 columns)  
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
```

```

# Create epoch range for x-axis
epochs = range(1, len(history['train_loss']) + 1)

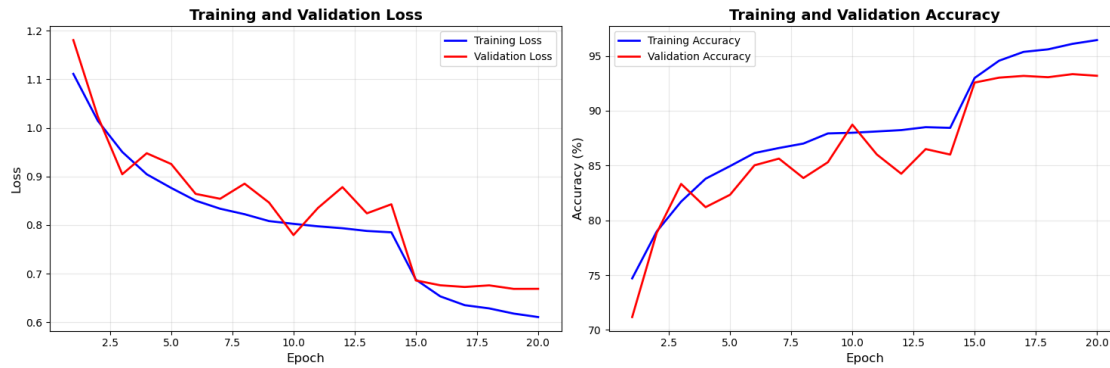
# Plot loss curves
# Plot training loss
ax1.plot(epochs, history['train_loss'], 'b-', label='Training Loss',
↪linewidth=2)
# Plot validation loss
ax1.plot(epochs, history['val_loss'], 'r-', label='Validation Loss',
↪linewidth=2)
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.set_title('Training and Validation Loss', fontsize=14,
↪fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot accuracy curves
# Plot training accuracy
ax2.plot(epochs, history['train_acc'], 'b-', label='Training Accuracy',
↪linewidth=2)
# Plot validation accuracy
ax2.plot(epochs, history['val_acc'], 'r-', label='Validation Accuracy',
↪linewidth=2)
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Accuracy (%)', fontsize=12)
ax2.set_title('Training and Validation Accuracy', fontsize=14,
↪fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3)

# Adjust layout to prevent overlap
plt.tight_layout()
plt.show()

# Plot training history
plot_training_history(history)

```



4 PART 4: EVALUATION

4.0.1 Test Set Evaluation

```
[16]: def evaluate_test_set(model, test_loader, criterion, device, model_path=None):
    """
    Evaluate model on test set.

    Loads best model weights and computes final test metrics.

    Args:
        model (nn.Module): Model to evaluate
        test_loader (DataLoader): Test data loader
        criterion: Loss function
        device: Device to use
        model_path (str): Path to saved model weights

    Returns:
        tuple: (test_loss, test_accuracy)
    """
    # Load best model weights if path provided
    if model_path:
        # Load state dictionary from file
        model.load_state_dict(torch.load(model_path))
        print(f" Loaded model weights from: {model_path}")

    # Set model to evaluation mode
    model.eval()

    # Initialize metrics
    test_loss = 0.0
    correct = 0
    total = 0
```

```

# Disable gradient computation
with torch.no_grad():
    # Create progress bar for tested batches
    test_pbar = tqdm(test_loader, desc='Testing')

    # Iterate through test batches
    for inputs, labels in test_pbar:
        # Move data to device
        inputs = inputs.to(device)
        labels = labels.to(device)

        # Forward pass
        outputs = model(inputs)
        # Compute loss
        loss = criterion(outputs, labels)

        # Accumulate metrics
        test_loss += loss.item() * inputs.size(0)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

        # Update progress bar
        test_pbar.set_postfix({
            'loss': f'{loss.item():.4f}',
            'acc': f'{100.0 * correct / total:.2f}%'
        })

    # Calculate final test metrics
    test_loss = test_loss / total
    test_acc = 100.0 * correct / total

    print("\n" + "=")
    print("TEST SET EVALUATION")
    print("="*80)
    print(f"Test Loss: {test_loss:.4f}")
    print(f"Test Accuracy: {test_acc:.2f}%")
    print("="*80)

    return test_loss, test_acc

# Evaluate on test set
test_loss, test_acc = evaluate_test_set(
    model=model,
    test_loader=test_loader,
    criterion=criterion,

```

```

device=device,
model_path='resnet50_cifar10_anti_overfitting.pth'
)

```

```

/var/folders/kd/j0t_qpxn5232l1_zzh1cb2wc0000gn/T/ipykernel_12951/2256378374.py:2
0: FutureWarning: You are using `torch.load` with `weights_only=False` (the
current default value), which uses the default pickle module implicitly. It is
possible to construct malicious pickle data which will execute arbitrary code
during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for
more details). In a future release, the default value for `weights_only` will be
flipped to `True`. This limits the functions that could be executed during
unpickling. Arbitrary objects will no longer be allowed to be loaded via this
mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting
`weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this
experimental feature.

```

```

    model.load_state_dict(torch.load(model_path))

```

```

Loaded model weights from: resnet50_cifar10_anti_overfitting.pth

```

```

Testing:   0%|          | 0/157 [00:00<?, ?it/s]

```

```

=

```

```

TEST SET EVALUATION

```

```

=====
Test Loss: 0.6750

```

```

Test Accuracy: 92.90%
=====

```

4.0.2 Confusion Matrix

```

[11]: from sklearn.metrics import confusion_matrix
import seaborn as sns

def plot_confusion_matrix(model, test_loader, class_names, device):
    """
    Compute and plot confusion matrix for test set.

    Shows which classes are confused with each other.

    Args:
        model (nn.Module): Trained model
        test_loader (DataLoader): Test data loader
        class_names (list): List of class names
        device: Device to use
    """

```

```

# Set model to evaluation mode
model.eval()
# Initialize lists for predictions and true labels
all_preds = []
all_labels = []

# Disable gradient computation
with torch.no_grad():
    # Iterate through test set
    for inputs, labels in tqdm(test_loader, desc='Computing predictions'):
        # Move data to device
        inputs = inputs.to(device)
        labels = labels.to(device)

        # Get predictions
        outputs = model(inputs)
        _, predicted = torch.max(outputs, 1)

        # Store predictions and labels (move to CPU)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Compute confusion matrix
cm = confusion_matrix(all_labels, all_preds)

# Create figure
plt.figure(figsize=(12, 10))
# Plot confusion matrix as heatmap
sns.heatmap(
    cm,
    annot=True, # Show numbers in cells
    fmt='d', # Integer format
    cmap='Blues', # Color scheme
    xticklabels=class_names, # X-axis labels
    yticklabels=class_names # Y-axis labels
)
plt.xlabel('Predicted Label', fontsize=12)
plt.ylabel('True Label', fontsize=12)
plt.title('Confusion Matrix - CIFAR-10 Test Set', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.show()

# Calculate per-class accuracy
print("\nPer-Class Accuracy:")
print("-" * 40)
for i, class_name in enumerate(class_names):

```

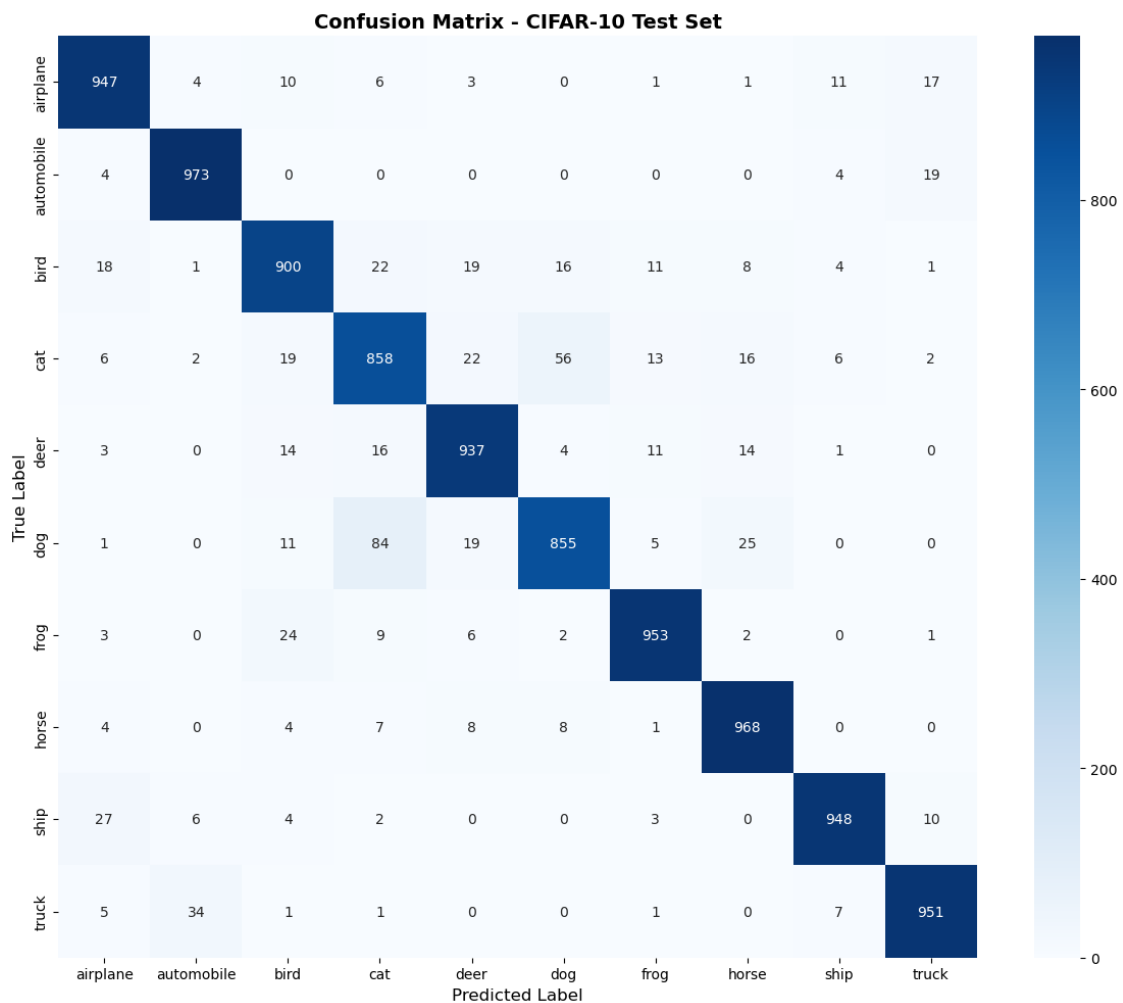
```

# Calculate accuracy for this class
class_correct = cm[i, i]
class_total = cm[i].sum()
class_acc = 100.0 * class_correct / class_total
print(f"{class_name:12s}: {class_acc:.2f}% ({class_correct}/
↪{class_total})")

# Plot confusion matrix
plot_confusion_matrix(model, test_loader, class_names, device)

```

Computing predictions: 0%| | 0/157 [00:00<?, ?it/s]



Per-Class Accuracy:

```

-----
airplane    : 94.70% (947/1000)
automobile  : 97.30% (973/1000)

```

```

bird      : 90.00% (900/1000)
cat       : 85.80% (858/1000)
deer      : 93.70% (937/1000)
dog       : 85.50% (855/1000)
frog      : 95.30% (953/1000)
horse     : 96.80% (968/1000)
ship      : 94.80% (948/1000)
truck     : 95.10% (951/1000)

```

4.0.3 Sample Predictions Visualization

```

[31]: def visualize_predictions(model, test_loader, class_names, device,
    ↪ num_samples=16):
    """
    Visualize model predictions on random test samples.

    Shows images with true labels and predicted labels.
    Green = correct, Red = incorrect.

    Args:
        model (nn.Module): Trained model
        test_loader (DataLoader): Test data loader
        class_names (list): List of class names
        device: Device to use
        num_samples (int): Number of samples to display
    """
    # Set model to evaluation mode
    model.eval()

    # Get a batch of test data
    dataiter = iter(test_loader)
    images, labels = next(dataiter)

    # Move data to device and get predictions
    images_device = images.to(device)
    # Disable gradient computation
    with torch.no_grad():
        outputs = model(images_device)
        _, predictions = torch.max(outputs, 1)

    # Denormalization constants
    mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
    std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)

    # Create grid of subplots (4x4)
    fig, axes = plt.subplots(4, 4, figsize=(16, 16))

```

```

fig.suptitle('Sample Predictions on Test Set', fontsize=16,
fontweight='bold', y=1.01)

# Display predictions for each sample
for idx in range(num_samples):
    # Calculate subplot position
    ax = axes[idx // 4, idx % 4]

    # Denormalize image
    img = images[idx] * std + mean
    img = torch.clamp(img, 0, 1)
    img = img.permute(1, 2, 0).numpy()

    # Get true and predicted labels
    true_label = class_names[labels[idx]]
    pred_label = class_names[predictions[idx].cpu()]

    # Determine color (green if correct, red if wrong)
    color = 'green' if true_label == pred_label else 'red'

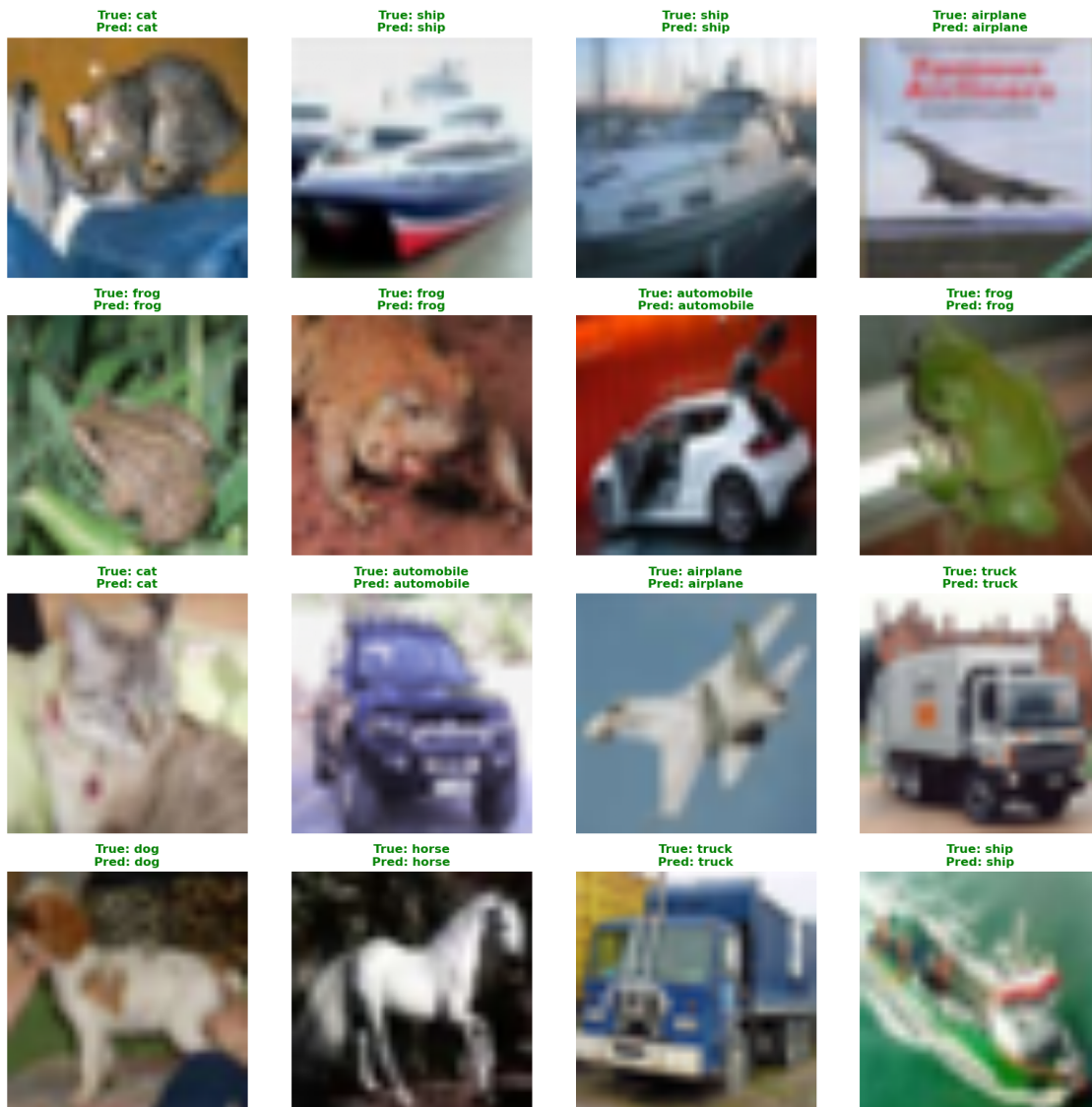
    # Display image
    ax.imshow(img)
    # Set title with true and predicted labels
    ax.set_title(f'True: {true_label}\nPred: {pred_label}',
                color=color, fontweight='bold')
    ax.axis('off')

plt.tight_layout()
plt.show()

# Visualize predictions
visualize_predictions(model, test_loader, class_names, device, num_samples=16)

```

Sample Predictions on Test Set



5 PART 5: EXPLAINABLE AI - GRAD-CAM

5.0.1 Grad-CAM Implementation

```
[18]: import torch.nn.functional as F
import cv2

class GradCAM:
    """
    Grad-CAM implementation for ResNet-50.
```

Algorithm:

- 1. Forward pass: Get model predictions*
- 2. Backward pass: Compute gradients for target class*
- 3. Weight feature maps by gradient importance*
- 4. Generate heatmap overlay*

"""

```
def __init__(self, model, target_layer):
```

"""

Initialize Grad-CAM.

Args:

model (nn.Module): Trained ResNet-50

target_layer (nn.Module): Layer to hook (model.layer4[-1] for ↪ResNet-50)

"""

```
self.model = model
```

```
self.target_layer = target_layer
```

```
self.gradients = None
```

```
self.activations = None
```

Register hooks to capture gradients and activations

```
self.target_layer.register_forward_hook(self._save_activation)
```

```
self.target_layer.register_full_backward_hook(self._save_gradient)
```

```
def _save_activation(self, module, input, output):
```

"""Hook to save forward pass activations."""

```
self.activations = output.detach()
```

```
def _save_gradient(self, module, grad_input, grad_output):
```

"""Hook to save backward pass gradients."""

```
self.gradients = grad_output[0].detach()
```

```
def generate_cam(self, input_image, target_class):
```

"""

Generate Grad-CAM heatmap for a single image.

Args:

input_image (torch.Tensor): Preprocessed image (1, 3, 224, 224)

target_class (int): Target class index (0-9 for CIFAR-10)

Returns:

np.ndarray: Heatmap (224, 224) with values [0, 1]

"""

Set model to evaluation mode

```
self.model.eval()
```

Forward pass

```
output = self.model(input_image)
```

```

        # Zero all gradients
        self.model.zero_grad()
        # Backward pass for target class
        output[0, target_class].backward()
        # Get gradients and activations
        gradients = self.gradients[0].cpu()
        activations = self.activations[0].cpu()
        # Global average pooling of gradients (importance weights)
        weights = torch.mean(gradients, dim=(1, 2))

        # Weighted sum of activations
        cam = torch.zeros(activations.shape[1:], dtype=torch.float32)
        for i, w in enumerate(weights):
            cam += w * activations[i, :, :]

        # Apply ReLU
        cam = F.relu(cam)
        # Normalize to [0,1]
        cam = cam - cam.min()
        if cam.max() > 0:
            cam = cam / cam.max()
        # Resize to input image (224x224)
        cam = cam.numpy()
        cam = cv2.resize(cam, (224, 224))
        return cam
print(" Grad-CAM class implemented successfully")

```

Grad-CAM class implemented successfully

6 PART 6: EXPLAINABLE AI - LIME

6.0.1 LIME Implementation

```

[19]: from lime import lime_image
      from skimage.segmentation import mark_boundaries

      def batch_predict(images, model, device):
          """
          Batch prediction function for LIME.

          Args:
              images (np.ndarray): Batch of images (N, H, W, C) in [0, 1]
              model: PyTorch model
              device: Computation device

          Returns:
              np.ndarray: Prediction probabilities (N, 10)
          """

```

```

"""
model.eval()

# Preprocessing: normalize with ImageNet stats
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])

# Convert to PyTorch format: (N, H, W, C) -> (N, C, H, W)
images_normalized = (images - mean) / std
images_tensor = torch.from_numpy(images_normalized).permute(0, 3, 1, 2).
↳float().to(device)

# Get predictions
with torch.no_grad():
    outputs = model(images_tensor)
    probs = F.softmax(outputs, dim=1)

    return probs.cpu().numpy()

print(" LIME prediction wrapper implemented successfully")

```

LIME prediction wrapper implemented successfully

7 PART 7: COMPARATIVE ANALYSIS

7.0.1 Grad-CAM vs. LIME Side-by-Side Comparison

```

[32]: def compare_gradcam_lime(model, test_loader, class_names, device,
↳class_indices):
    """
    Side-by-side comparison of Grad-CAM and LIME for selected CIFAR-10 classes.

    This function visualizes, for each selected class:
    1. The original image
    2. Grad-CAM heatmap overlay
    3. LIME superpixel boundaries
    4. Combined overlay (Grad-CAM + LIME contours)

    Args:
    model (nn.Module): Trained model
    test_loader (DataLoader): Test data loader
    class_names (list): List of CIFAR-10 class names
    device: Computation device (CPU/CUDA/MPS)
    class_indices (list): List of class indices to visualize (e.g.
↳[0,1,2,3,4])
    """

    # Initialize Grad-CAM and LIME explainer

```

```

grad_cam = GradCAM(model, target_layer=model.layer4[-1])
lime_explainer = lime_image.LimeImageExplainer()

# ImageNet normalization constants for denormalization
mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)

# --- STEP 1: Collect one sample per class ---
# Dictionary to hold one (image, label) tuple for each class
class_samples = {i: None for i in range(10)}
for images, labels in test_loader:
    for idx, label in enumerate(labels):
        class_idx = label.item()
        # Store only the first sample found for each class
        if class_samples[class_idx] is None:
            class_samples[class_idx] = (images[idx], label)
    # Stop once all classes have a sample
    if all(sample is not None for sample in class_samples.values()):
        break

# --- STEP 2: Create visualization grid ---
num_samples = len(class_indices)
fig, axes = plt.subplots(num_samples, 4, figsize=(16, num_samples * 4))
fig.suptitle('Grad-CAM vs. LIME Comparison', fontsize=16,
fontweight='bold', y=1.01)

# --- STEP 3: Process each selected class ---
for row_idx, class_idx in enumerate(class_indices):
    img_tensor, true_label = class_samples[class_idx]

    # Prepare image for model and visualization
    img_tensor_device = img_tensor.unsqueeze(0).to(device)
    img_denorm = img_tensor * std + mean # Denormalize
    img_denorm = torch.clamp(img_denorm, 0, 1)
    img_np = img_denorm.permute(1, 2, 0).numpy() # Convert to HWC numpy

    # --- Model prediction ---
    model.eval()
    with torch.no_grad():
        output = model(img_tensor_device)
        pred_class = output.argmax(dim=1).item()
        confidence = torch.softmax(output, dim=1)[0, pred_class].item()

    # --- Grad-CAM heatmap ---
    cam = grad_cam.generate_cam(img_tensor_device, pred_class)

    # --- LIME explanation ---

```

```

lime_explanation = lime_explainer.explain_instance(
    img_np,
    lambda x: batch_predict(x, model, device),
    top_labels=1,
    hide_color=0,
    num_samples=1000
)
temp, mask = lime_explanation.get_image_and_mask(
    pred_class,
    positive_only=True,
    num_features=10,
    hide_rest=False
)

# --- Visualization ---
# Color for title: green if correct, red if incorrect
correct = (class_idx == pred_class)
title_color = 'green' if correct else 'red'

# Column 1: Original image + prediction info
axes[row_idx, 0].imshow(img_np)
axes[row_idx, 0].set_title(
    f'{class_names[class_idx]}\nPred: {class_names[pred_class]}\nConf:␣
↪{confidence:.2f}',
    color=title_color,
    fontsize=10,
    fontweight='bold'
)
axes[row_idx, 0].axis('off')

# Column 2: Grad-CAM overlay
axes[row_idx, 1].imshow(img_np)
axes[row_idx, 1].imshow(cam, cmap='jet', alpha=0.5)
if row_idx == 0:
    axes[row_idx, 1].set_title('Grad-CAM', fontsize=12,␣
↪fontweight='bold')
axes[row_idx, 1].axis('off')

# Column 3: LIME superpixel boundaries
axes[row_idx, 2].imshow(mark_boundaries(temp, mask))
if row_idx == 0:
    axes[row_idx, 2].set_title('LIME', fontsize=12, fontweight='bold')
axes[row_idx, 2].axis('off')

# Column 4: Combined overlay (Grad-CAM + LIME contours)
axes[row_idx, 3].imshow(img_np)
axes[row_idx, 3].imshow(cam, cmap='jet', alpha=0.3)

```

```

        axes[row_idx, 3].contour(mask, colors='green', linewidths=2)
        if row_idx == 0:
            axes[row_idx, 3].set_title('Combined', fontsize=12,
↪fontweight='bold')
            axes[row_idx, 3].axis('off')

        # --- Final layout and display ---
        plt.tight_layout()
        plt.show()
        print("\n Visualization complete for selected classes")

# Usage example:
# Show first 5 classes
compare_gradcam_lime(model, test_loader, class_names, device,
↪class_indices=[0,1,2,3,4])
# Show last 5 classes
compare_gradcam_lime(model, test_loader, class_names, device,
↪class_indices=[5,6,7,8,9])

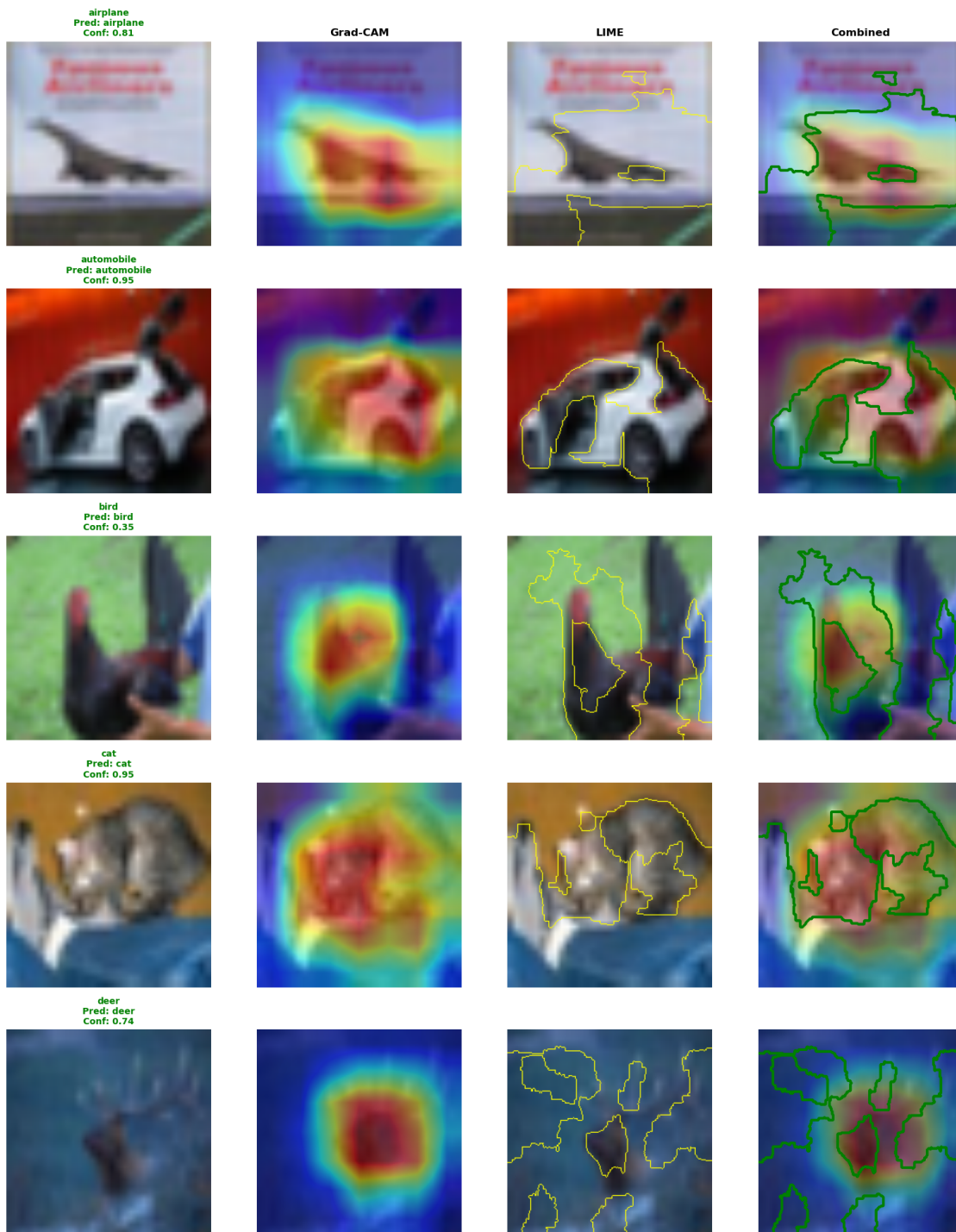
```

```

0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]

```

Grad-CAM vs. LIME Comparison



Visualization complete for selected classes

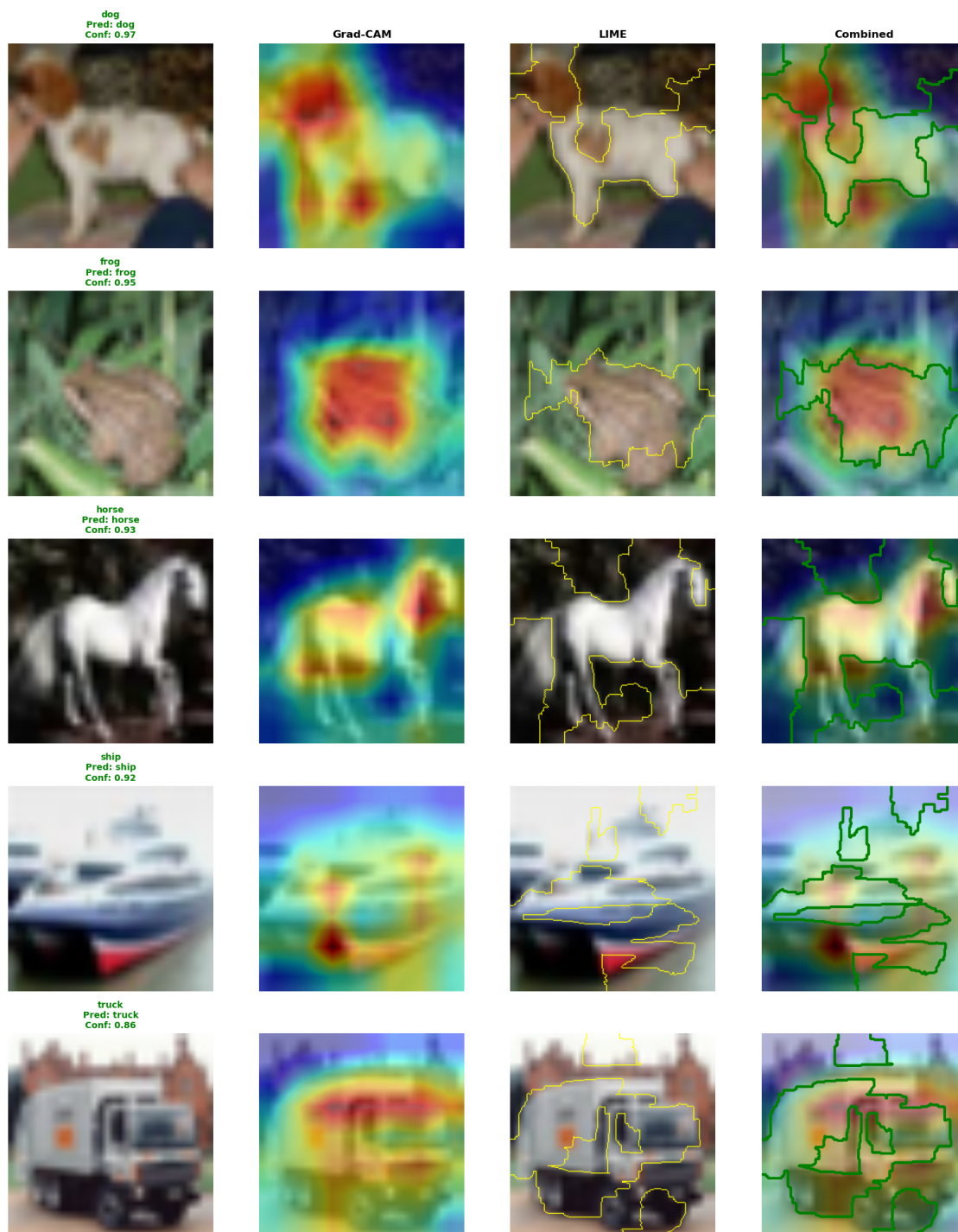
0% | 0/1000 [00:00<?, ?it/s]

```

0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]

```

Grad-CAM vs. LIME Comparison



Visualization complete for selected classes

7.0.2 Quantitative Comparison Metrics

```
[29]: def compute_quantitative_metrics_stratified(model, test_loader, class_names,
        ↪device, samples_per_class=2):
        """
        Compute quantitative metrics with STRATIFIED sampling (guaranteed coverage
        ↪of all classes).

        Ensures 2 samples per class → 20 total samples covering all 10 CIFAR-10
        ↪classes.

        Args:
            model (nn.Module): Trained model
            test_loader (DataLoader): Test data
            class_names (list): CIFAR-10 class names
            device: Computation device
            samples_per_class (int): Number of samples per class (default: 2)
        """
        # Initialize
        grad_cam = GradCAM(model, target_layer=model.layer4[-1])
        lime_explainer = lime_image.LimeImageExplainer()

        # Denormalization constants
        mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
        std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)

        # STEP 1: Collect samples for each class (stratified sampling)
        print("Collecting stratified samples (2 per class)...")
        class_samples = {i: [] for i in range(10)}

        for images, labels in test_loader:
            for idx, label in enumerate(labels):
                class_idx = label.item()
                # Only collect if we need more samples for this class
                if len(class_samples[class_idx]) < samples_per_class:
                    class_samples[class_idx].append((images[idx], label))

            # Stop once ALL classes have enough samples
            if all(len(samples) >= samples_per_class for samples in class_samples.
            ↪values()):
                print(" Found 2 samples for all 10 classes!")
                break
```

```

# Verify coverage
print("\nSample distribution:")
for class_idx in range(10):
    print(f" {class_names[class_idx]:12s}: {len(class_samples[class_idx])}
↪samples")

# STEP 2: Compute IoU for collected samples
iou_scores = []
class_iou = {i: [] for i in range(10)}

print("\nComputing IoU for stratified samples...")
for class_idx in tqdm(range(10), desc='Classes'):
    for img_tensor, true_label in class_samples[class_idx]:
        # Prepare image
        img_tensor_device = img_tensor.unsqueeze(0).to(device)
        img_denorm = img_tensor * std + mean
        img_denorm = torch.clamp(img_denorm, 0, 1)
        img_np = img_denorm.permute(1, 2, 0).numpy()

        # Get prediction
        model.eval()
        with torch.no_grad():
            output = model(img_tensor_device)
            pred_class = output.argmax(dim=1).item()

        # Generate Grad-CAM (binarize top 30% pixels)
        cam = grad_cam.generate_cam(img_tensor_device, pred_class)
        gradcam_mask = (cam > np.percentile(cam, 70)).astype(int)

        # Generate LIME
        lime_explanation = lime_explainer.explain_instance(
            img_np,
            lambda x: batch_predict(x, model, device),
            top_labels=1,
            hide_color=0,
            num_samples=1000
        )
        _, lime_mask = lime_explanation.get_image_and_mask(
            pred_class,
            positive_only=True,
            num_features=10,
            hide_rest=False
        )
        lime_mask = lime_mask.astype(int)

# Compute IoU

```

```

        intersection = np.logical_and(gradcam_mask, lime_mask).sum()
        union = np.logical_or(gradcam_mask, lime_mask).sum()
        iou = intersection / union if union > 0 else 0

        iou_scores.append(iou)
        class_iou[class_idx].append(iou)

# Print results
print("\n" + "="*80)
print("QUANTITATIVE COMPARISON METRICS (STRATIFIED)")
print("="*80)

print(f"\nOverall Statistics:")
print(f"  Mean IoU: {np.mean(iou_scores):.3f}")
print(f"  Median IoU: {np.median(iou_scores):.3f}")
print(f"  Std Dev: {np.std(iou_scores):.3f}")
print(f"  Min IoU: {np.min(iou_scores):.3f}")
print(f"  Max IoU: {np.max(iou_scores):.3f}")

print(f"\nPer-Class IoU (ALL 10 CLASSES):")
print("-" * 40)
for class_idx in range(10):
    mean_iou = np.mean(class_iou[class_idx])
    print(f"{class_names[class_idx]:12s}: {mean_iou:.3f}")
↪(n={len(class_iou[class_idx])})")

# Visualize IoU distribution
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# Histogram of IoU scores
ax1.hist(iou_scores, bins=20, edgecolor='black', alpha=0.7)
ax1.axvline(np.mean(iou_scores), color='red', linestyle='--', linewidth=2,
            label=f'Mean: {np.mean(iou_scores):.3f}')
ax1.set_xlabel('IoU Score', fontsize=12)
ax1.set_ylabel('Frequency', fontsize=12)
ax1.set_title('Distribution of Spatial Overlap (IoU)', fontsize=14,
↪fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Per-class IoU bar chart (NOW WITH ALL 10 CLASSES!)
class_means = [np.mean(class_iou[i]) for i in range(10)]
bars = ax2.bar(range(10), class_means, edgecolor='black', alpha=0.7)

# Color bars by IoU level
for i, bar in enumerate(bars):
    if class_means[i] > 0.40:

```

```

        bar.set_color('green')
    elif class_means[i] > 0.30:
        bar.set_color('orange')
    else:
        bar.set_color('red')

ax2.set_xticks(range(10))
ax2.set_xticklabels(class_names, rotation=45, ha='right')
ax2.set_xlabel('Class', fontsize=12)
ax2.set_ylabel('Mean IoU', fontsize=12)
ax2.set_title('Per-Class Spatial Overlap (ALL CLASSES)', fontsize=14,
fontweight='bold')
ax2.axhline(0.40, color='green', linestyle='--', alpha=0.5, label='High (>0.
40)')
ax2.axhline(0.30, color='orange', linestyle='--', alpha=0.5,
label='Moderate (0.30-0.40)')
ax2.legend()
ax2.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# Interpretation
print("\n" + "="*80)
print("INTERPRETATION")
print("="*80)
if np.mean(iou_scores) > 0.4:
    print(" HIGH AGREEMENT: Grad-CAM and LIME show strong spatial overlap")
    print(" Both methods focus on similar image regions")
elif np.mean(iou_scores) > 0.25:
    print("~ MODERATE AGREEMENT: Grad-CAM and LIME show partial overlap")
    print(" Methods agree on some regions but differ in specifics")
else:
    print(" LOW AGREEMENT: Grad-CAM and LIME show different focus regions")
    print(" Methods may be capturing different aspects of the decision")

# Class-specific insights
print("\n" + "="*80)
print("CLASS-SPECIFIC INSIGHTS")
print("="*80)

high_agreement = [i for i in range(10) if np.mean(class_iou[i]) > 0.40]
moderate_agreement = [i for i in range(10) if 0.30 <= np.mean(class_iou[i])
<= 0.40]
low_agreement = [i for i in range(10) if np.mean(class_iou[i]) < 0.30]

if high_agreement:

```

```

print(f"\n HIGH AGREEMENT (IoU > 0.40):")
for idx in high_agreement:
    print(f"    • {class_names[idx]:12s}: {np.mean(class_iou[idx]):.3f}")
print(" → These classes have stable, discriminative features")

if moderate_agreement:
    print(f"\n MODERATE AGREEMENT (IoU 0.30-0.40):")
    for idx in moderate_agreement:
        print(f"    • {class_names[idx]:12s}: {np.mean(class_iou[idx]):.3f}")
    print(" → These classes use feature ensembles or context")

if low_agreement:
    print(f"\n LOW AGREEMENT (IoU < 0.30):")
    for idx in low_agreement:
        print(f"    • {class_names[idx]:12s}: {np.mean(class_iou[idx]):.3f}")
    print(" → These classes lack stable features (model uncertain)")

print("="*80)

# Run stratified analysis
print("\n" + "="*80)
print("RUNNING STRATIFIED QUANTITATIVE ANALYSIS")
print("="*80)
compute_quantitative_metrics_stratified(model, test_loader, class_names,
    ↪device, samples_per_class=2)

```

```

=====
RUNNING STRATIFIED QUANTITATIVE ANALYSIS
=====

```

Collecting stratified samples (2 per class)...

Found 2 samples for all 10 classes!

Sample distribution:

```

airplane      : 2 samples
automobile    : 2 samples
bird          : 2 samples
cat           : 2 samples
deer          : 2 samples
dog           : 2 samples
frog          : 2 samples
horse         : 2 samples
ship          : 2 samples
truck         : 2 samples

```

Computing IoU for stratified samples...

Classes: 0%| | 0/10 [00:00<?, ?it/s]

```

0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]
0%|          | 0/1000 [00:00<?, ?it/s]

```

```

=====
QUANTITATIVE COMPARISON METRICS (STRATIFIED)
=====

```

Overall Statistics:

```

Mean IoU: 0.392
Median IoU: 0.347
Std Dev: 0.095
Min IoU: 0.282
Max IoU: 0.643

```

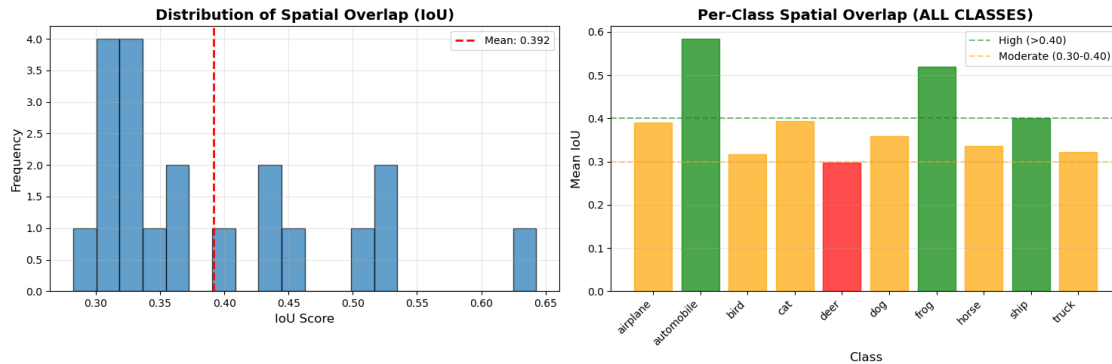
Per-Class IoU (ALL 10 CLASSES):

```

-----
airplane      : 0.390 (n=2)
automobile    : 0.584 (n=2)
bird          : 0.317 (n=2)
cat           : 0.394 (n=2)

```

deer : 0.298 (n=2)
 dog : 0.358 (n=2)
 frog : 0.519 (n=2)
 horse : 0.336 (n=2)
 ship : 0.401 (n=2)
 truck : 0.322 (n=2)



INTERPRETATION

~ MODERATE AGREEMENT: Grad-CAM and LIME show partial overlap
 Methods agree on some regions but differ in specifics

CLASS-SPECIFIC INSIGHTS

HIGH AGREEMENT (IoU > 0.40):

- automobile : 0.584
- frog : 0.519
- ship : 0.401

→ These classes have stable, discriminative features

MODERATE AGREEMENT (IoU 0.30-0.40):

- airplane : 0.390
- bird : 0.317
- cat : 0.394
- dog : 0.358
- horse : 0.336
- truck : 0.322

→ These classes use feature ensembles or context

LOW AGREEMENT (IoU < 0.30):

- deer : 0.298

→ These classes lack stable features (model uncertain)

=====